

OE-EC804A: Artificial Intelligence

Complete Lecture Notes, solved MAKAUT previous year questions, and active recall study modules.

Semester VIII Notes
Prepared by: Firdous Rahaman

June 12, 2026

Contents

1	Introduction to Artificial Intelligence	2
1.1	Section 1: Definition and Four Approaches to AI (Q1.1)	2
1.1.1	1. Definition of Artificial Intelligence (AI)	2
1.1.2	2. The Four Conceptual Approaches to AI	2
1.2	Section 2: The Turing Test Benchmark (Q1.2)	3
1.2.1	1. Definition and Setup	3
1.2.2	2. Six Key Capabilities Required to Pass the Test	4
1.3	Section 3: Foundational Disciplines of AI (Q1.3)	4
1.4	Section 4: Components and Sub-fields of AI (Q1.4)	5
1.4.1	1. Functional Sub-fields of AI	5
1.5	Section 5: Expert Systems (Q1.5)	5
1.5.1	1. Definition	5
1.5.2	2. Detailed Architecture of an Expert System	6
1.5.3	3. Advantages and Disadvantages of Expert Systems	6
2	Intelligent Agents	8
2.1	Section 1: Definition of Agent, Environment, and Percept Sequence (Q2.1)	8
2.1.1	1. Definitions	8
2.1.2	2. Relationship and Core Mathematical Formulation	8
2.2	Section 2: PEAS Description (Q2.2)	9
2.2.1	1. Definition of PEAS	9
2.2.2	2. Comparative PEAS Examples for Standard Systems	10
2.3	Section 3: The Five Basic Agent Structures (Q2.3)	10
2.3.1	1. Simple Reflex Agents	10
2.3.2	2. Model-Based Reflex Agents	11
2.3.3	3. Goal-Based Agents	11
2.3.4	4. Utility-Based Agents	12
2.3.5	5. Learning Agents	12
2.4	Section 4: Nature of Environments (Q2.4)	13
2.4.1	1. Environment Properties Explained	13
2.4.2	2. Comparison Matrix of Common Environments	14
3	Solving Problems by Searching (Uninformed Search)	15
3.1	Section 1: Problem-Solving Agents and Formulation (Q3.1)	15
3.1.1	1. Definition of Problem-Solving Agents	15
3.1.2	2. Steps in Problem Formulation	15
3.1.3	3. State Space Representation	15
3.2	Section 2: Evaluation Criteria for Search Algorithms (Q3.2)	16
3.2.1	Key Parameters:	16
3.3	Section 3: BFS vs. DFS Comparison (Q3.3)	16
3.3.1	1. Feature Comparison Table	16

3.3.2	2. Why DFS is Not Always Complete	16
3.4	Section 4: IDDFS and UCS Algorithms (Q3.4)	17
3.4.1	1. Iterative Deepening Depth-First Search (IDDFS)	17
3.4.2	2. Uniform-Cost Search (UCS)	17
3.5	Section 5: Formulations of Classic Puzzles (Q3.5)	18
3.5.1	1. 8-Puzzle Problem Formulation	18
3.5.2	2. The Missionaries and Cannibals Problem (3M, 3C)	18
3.5.3	3. The Water Jug Problem (4-Gal and 3-Gal Jugs)	19
4	Informed Search & Exploration	22
4.1	Section 1: Greedy Best-First Search vs. A* Search (Q4.1)	22
4.1.1	1. Comparison Table	22
4.1.2	2. Conditions for A* Optimality and Completeness	22
4.2	Section 2: Admissibility and Consistency of Heuristics (Q4.2)	22
4.2.1	1. Admissibility	22
4.2.2	2. Consistency (Monotonicity)	23
4.2.3	3. Proof: Consistent Heuristic is Admissible	23
4.3	Section 3: Simulated Annealing Search (Q4.3)	23
4.3.1	1. Working Principle	23
4.3.2	2. Algorithm Steps	24
4.3.3	3. Impact of the Cooling Schedule	24
4.4	Section 4: Mathematical Proofs in Informed Search (Q4.4)	24
4.4.1	1. Proof: A* Tree Search is Admissible (Optimal)	24
4.4.2	2. Proof: Consistent Heuristic implies Monotonic f-values	25
4.5	Section 5: A* Search Solving the 8-Puzzle (Q4.5)	25
4.5.1	1. Heuristic Selection	25
4.5.2	2. Trace Example: Initial State to Goal State	26
4.6	Section 6: Local Search and Optimization Techniques (Q4.6)	28
4.6.1	1. Hill-Climbing Search	28
4.6.2	2. Local Beam Search	28
4.6.3	3. AO* Search (AND-OR Graph Search)	28
4.7	Section 7: Informed Search Properties & Linear Space Search (Q4.7) [5M]	29
4.7.1	1. Heuristic Search Concept & Greedy Expansion	29
4.7.2	2. Space Complexity of Greedy Search	29
4.7.3	3. Recursive Best-First Search (RBFS)	30
5	Constraint Satisfaction Problems	31
5.1	Section 1: Introduction to Constraint Satisfaction Problems (Q5.1)	31
5.1.1	1. Definition of a CSP	31
5.1.2	2. Map-Coloring Example	31
5.2	Section 2: Solving Cryptarithmic Problems (Q5.2)	32
5.2.1	1. Problem Formulation	32
5.2.2	2. Step-by-Step Deduction Process	33
5.2.3	3. Final Solution Values	34
5.3	Section 3: Flexible CSPs, Solvers, and Constraint Programming (Q5.3) [5M]	35
5.3.1	1. Flexible CSPs	35
5.3.2	2. Backtracking Search for Finite Domain CSPs	35
5.3.3	3. Languages for Constraint Programming	35

6	Adversarial Search	37
6.1	Section 1: Minimax Game-Playing Procedure (Q6.1)	37
6.1.1	1. Definition and Working Principle	37
6.1.2	2. Time and Space Complexity	38
6.2	Section 2: Alpha-Beta Pruning (Q6.2)	38
6.2.1	1. Working Principle	38
6.2.2	2. Trace Example: 3-Level Branching Game Tree	38
7	Logical Agents	41
7.1	Section 1: Propositional Logic Definitions and Tautology Proof (Q7.1)	41
7.1.1	1. Key Definitions in Propositional Logic	41
7.1.2	2. Proof: $((P \rightarrow Q) \rightarrow P) \rightarrow P$ is a Tautology	41
7.2	Section 2: Logical Inference Rules (Q7.2)	42
7.2.1	1. Modus Ponens	42
7.2.2	2. Modus Tollens	42
7.2.3	3. Resolution Rule	42
7.3	Section 3: Knowledge-Based Agents and Wumpus World (Q7.3)	43
7.3.1	1. Knowledge-Based Agents	43
7.3.2	2. The Wumpus World Environment	43
7.3.3	3. Step-by-Step Safe Logic Reasoning Example	44
7.4	Section 4: Propositional Semantics, Logical Inference, and Resolution (Q7.4) [5M]	44
7.4.1	1. Semantics and Proposition Symbols	45
7.4.2	2. Logical Inference Properties	45
7.4.3	3. CNF Equivalence & Clause Terminology	45
7.4.4	4. Resolution as a Single Inference Rule	45
8	First-Order Logic & Knowledge Representation	47
8.1	Section 1: Declarative vs. Procedural Knowledge and Production Systems (Q8.1)	47
8.1.1	1. Comparison of Knowledge Types	47
8.1.2	2. Production Systems in Knowledge Representation	47
8.2	Section 2: Horn Clauses and Skolemisation (Q8.2)	48
8.2.1	1. Horn Clause	48
8.2.2	2. Skolemisation	48
8.3	Section 3: PROLOG Programming (Q8.3)	48
8.3.1	1. Recursive Factorial Program	48
8.3.2	2. Recursive Greatest Common Divisor (GCD) Program	49
8.4	Section 4: Clausal Form (CNF) and Resolution Refutation Proofs (Q8.4)	49
8.4.1	1. Step-by-Step Procedure to Convert to CNF	49
8.4.2	2. CNF Conversion Example	49
8.4.3	3. Resolution Refutation Proof Case Study	50
8.5	Section 5: Predicate Logic Case Study – Marcus Hated Caesar (Q8.5)	51
8.5.1	1. FOPL Representation	51
8.5.2	2. Resolution Refutation Proof	51
8.6	Section 6: Knowledge Representation Techniques (Q8.6)	51
8.6.1	1. Semantic Networks	51
8.6.2	2. Frames	52
8.6.3	3. Fuzzy Logic	52
8.6.4	4. Neural Networks and Genetic Algorithms	53

9	Quick Revision: OE-EC804A - Artificial Intelligence	54
9.1	Unit I: Introduction	54
9.1.1	1. Important Definitions	54
9.1.2	Key Formulas	54
9.1.3	Frequently Asked Questions	55
9.1.4	Diagrams to Practice	55
9.1.5	One-Line Revision	56
9.2	Unit II: Intelligent Agents	56
9.2.1	1. Important Definitions	56
9.2.2	Key Formulas	57
9.2.3	Frequently Asked Questions	57
9.2.4	Diagrams to Practice	57
9.2.5	One-Line Revision	58
9.3	Unit III: Solving Problems by Searching (Uninformed Search)	59
9.3.1	1. Important Definitions	59
9.3.2	2. Characteristics of Uninformed Search	59
9.3.3	3. Important Search Algorithms	59
9.3.4	4. Algorithm Properties	60
9.3.5	Diagrams to Practice	60
9.4	Unit IV: Informed Search & Exploration	61
9.4.1	1. Important Definitions	61
9.4.2	2. Important Algorithms	61
9.4.3	Diagrams to Practice	61
9.4.4	One-Line Revision	62
9.5	Unit V: Constraint Satisfaction Problems	63
9.5.1	1. Important Definitions	63
9.5.2	Frequently Asked Questions	63
9.5.3	Diagrams to Practice	63
9.5.4	One-Line Revision	63
9.6	Unit VI: Adversarial Search	64
9.6.1	1. Important Definitions	64
9.6.2	Frequently Asked Questions	64
9.6.3	Diagrams to Practice	64
9.6.4	One-Line Revision	64
9.7	Unit VII: Logical Agents	64
9.7.1	1. Important Definitions	64
9.7.2	Frequently Asked Questions	65
9.7.3	Diagrams to Practice	65
9.7.4	One-Line Revision	65
9.8	Unit VIII: First-Order Logic & Knowledge Representation	66
9.8.1	1. Important Definitions	66
9.8.2	Frequently Asked Questions	66
9.8.3	Diagrams to Practice	66
9.8.4	One-Line Revision	67
9.9	Solved Exam One-Liners (CA4 Database)	67

Chapter 1

Introduction to Artificial Intelligence

1.1 Section 1: Definition and Four Approaches to AI (Q1.1)

1.1.1 1. Definition of Artificial Intelligence (AI)

Artificial Intelligence is a branch of Computer Science and Engineering concerned with building smart, autonomous systems capable of performing tasks that typically require human intelligence, such as visual perception, decision-making, learning, reasoning, and natural language understanding.

1.1.2 2. The Four Conceptual Approaches to AI

Historically, researchers have defined AI along two dimensions: **thought processes vs. behavior**, and **human-like performance vs. ideal rationality**. This yields four distinct conceptual views:

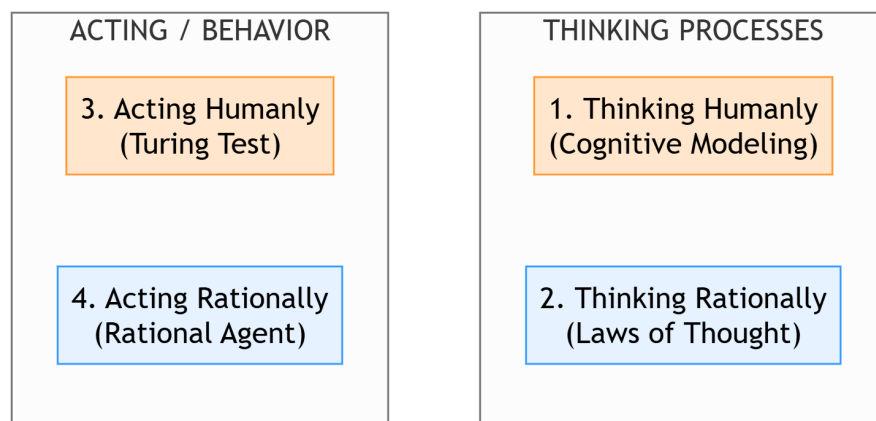


Figure 1.1: Four Conceptual Approaches to AI Matrix

Detailed Comparison of the Four Views:

Conceptual Approach	Core Philosophy / Focus	Methodology & Realization
1. Thinking Humanly	Focuses on mimicking the inner mental processing and cognitive activities of the human brain.	Implemented via Cognitive Science and neuroimaging. Systems are tested by comparing their decision steps to human experimental data.
2. Thinking Rationally	Focuses on using formal logic and the mathematical laws of thought to achieve "correct" reasoning.	Uses Propositional/First-Order Logic and syllogisms (e.g., "If A implies B, and A is true, then B is true").
3. Acting Humanly	Focuses on creating machines that perform physical and cognitive actions indistinguishable from a human.	Evaluated via the Turing Test . The system's inner logic is irrelevant; only the external, observable behavior matters.
4. Acting Rationally	Focuses on building Rational Agents that act to achieve the best possible outcome (or best expected outcome under uncertainty).	Realized through decision theory, search algorithms, and control theory. This is the standard modern view of AI.

1.2 Section 2: The Turing Test Benchmark (Q1.2)**1.2.1 1. Definition and Setup**

Introduced by **Alan Turing** in 1950, the **Turing Test** (originally the *Imitation Game*) is an operational test of machine intelligence.

- **Setup:** A human interrogator communicates with one human and one computer via typed text terminals in separate rooms.
- **Success Criterion:** If the interrogator cannot reliably tell the computer apart from the human after 5 minutes of questioning, the machine passes the test.

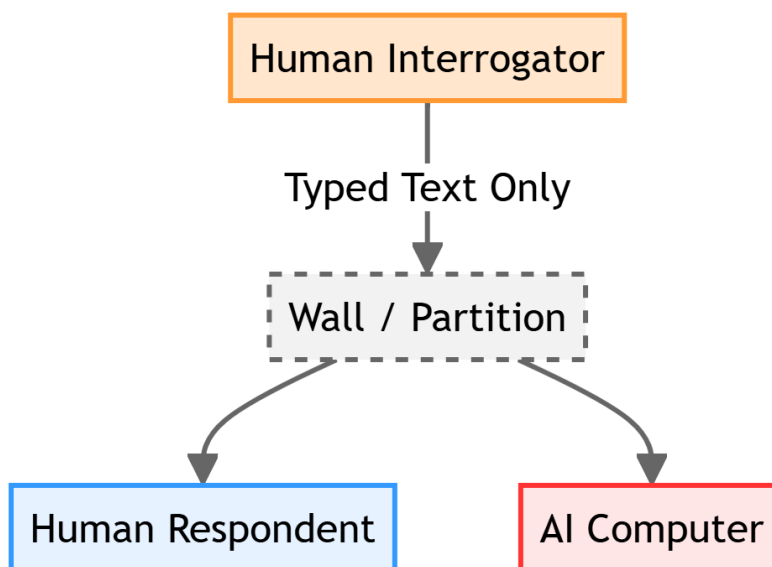


Figure 1.2: Turing Test Setup

1.2.2 2. Six Key Capabilities Required to Pass the Test

To pass the Turing Test, a machine must possess the following six primary sub-fields of AI:

1. **Natural Language Processing (NLP):** To communicate successfully in a natural language (like English).
2. **Knowledge Representation:** To store facts and beliefs acquired before or during the conversation.
3. **Automated Reasoning:** To use stored information to answer questions, draw conclusions, and find logical connections.
4. **Machine Learning:** To adapt to new circumstances, detect patterns, and learn from experience.
5. **Computer Vision** (*Total Turing Test requirement*): To perceive physical objects and actions of the interrogator.
6. **Robotics** (*Total Turing Test requirement*): To physically move, interact with, and manipulate the physical environment.

1.3 Section 3: Foundational Disciplines of AI (Q1.3)

Artificial Intelligence is a highly interdisciplinary field. The major contributing disciplines include:

1. Philosophy (400 BC – Present):

Contribution:* Formulated the ideas that the mind is like a machine, that knowledge is represented in formal languages, and that logical rules can be used to derive conclusions.

1. Mathematics (c. 800 – Present):

Contribution:* Provided the tools of **formal logic** (Boolean algebra), **probability theory** (handling uncertainty), and **algorithms** (computation limits and tractability).

1. Economics (1776 – Present):

Contribution:* Contributed utility theory, decision theory, and game theory, which govern how a rational agent makes choices to maximize its expected payoff.

1. Neuroscience (1861 – Present):

Contribution:* Mapped out how the physical brain processes information, laying the biological foundation for **Artificial Neural Networks (ANNs)**.

1. Psychology / Cognitive Science (1879 – Present):

Contribution:* Provided cognitive models to understand how humans think, perceive, and act, guiding the design of human-centric AI systems.

1. Control Theory & Cybernetics (1948 – Present):

Contribution:* Contributed feedback loops and the mathematical models to design adaptive, self-regulating systems (e.g., PID controllers, homeostatic systems).

1.4 Section 4: Components and Sub-fields of AI (Q1.4)

1.4.1 1. Functional Sub-fields of AI

AI is comprised of several interacting components, each addressing a specific dimension of intelligence:

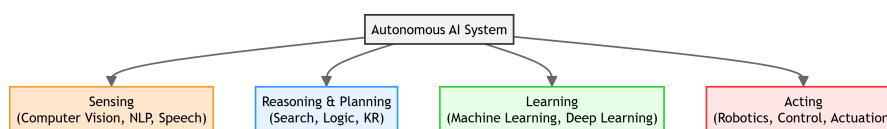


Figure 1.3: Functional Sub-fields/Components of AI

- **Computer Vision:** Enables machines to extract, process, and understand structured information from digital images or videos.
- **Natural Language Processing (NLP):** Enables computers to process, analyze, and synthesize human languages (syntax, semantics, and pragmatics).
- **Machine Learning (ML):** The core engine that allows algorithms to improve their performance automatically on a task through exposure to training data.
- **Expert Systems:** Specialized software systems that emulate the decision-making ability of a human expert in a narrow domain.
- **Automated Planning & Decision Making:** Search and logical reasoning systems designed to select sequences of actions to reach a desired goal state.
- **Robotics:** Integrates vision, control theory, and machine learning to build physical machines that can manipulate objects and navigate environments autonomously.

1.5 Section 5: Expert Systems (Q1.5)

1.5.1 1. Definition

An **Expert System (ES)** is an interactive, computer-based decision-making system that uses knowledge and inference procedures to solve complex, domain-specific problems that typically require human expertise (e.g., medical diagnosis, financial analysis, mineral exploration).

1.5.2 2. Detailed Architecture of an Expert System

A standard Expert System consists of three main non-overlapping components:

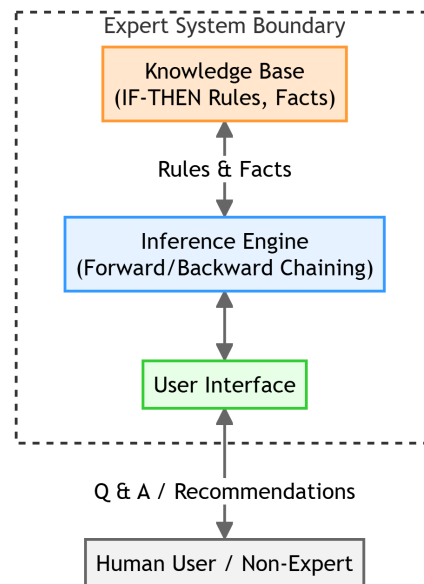


Figure 1.4: Expert System Architecture

Core Components Explained:

1. **Knowledge Base (KB):** The warehouse of high-quality domain-specific facts, rules, and heuristics compiled from human experts. Information is typically stored in the form of **IF-THEN production rules**.
2. **Inference Engine:** The processing engine that applies logical rules to the Knowledge Base to deduce new information or solve the user's query. It operates in two main modes:
 - **Forward Chaining (Data-Driven):** Starts with known facts and applies rules to infer new facts (reaches a conclusion).
 - **Backward Chaining (Goal-Driven):** Starts with a goal or hypothesis and works backward to find supporting facts.
1. **User Interface:** The communication interface that allows a non-expert user to input queries, view recommendations, and inspect the system's reasoning path (via an **Explanation Facility**).

1.5.3 3. Advantages and Disadvantages of Expert Systems

Advantages (At least 5 points):

1. **High Availability:** Expert knowledge is accessible 24/7 on ordinary computer systems, removing dependency on scarce human experts.
2. **Consistency:** Does not suffer from fatigue, emotional stress, or cognitive bias; always applies rules uniformly.
3. **Speed & Efficiency:** Processes large volumes of complex, rule-based data in milliseconds, reducing diagnosis or lookup times.

4. **Preservation of Expertise:** Captures and preserves the invaluable knowledge of senior experts before they retire or leave an organization.
5. **Explanatory Capability:** The system can explicitly print out its exact step-by-step reasoning path (Why/How a conclusion was reached), increasing user trust.

Disadvantages (At least 5 points):

1. **Lack of Common Sense:** Completely lacks general, everyday common-sense knowledge; cannot adapt if a problem falls slightly outside its narrow domain.
2. **High Maintenance Costs:** Domain knowledge changes over time, requiring expensive manual updates and validation of the rules database.
3. **Knowledge Acquisition Bottleneck:** Extracting tacit knowledge from human experts and translating it into rigid IF-THEN rules is extremely slow and difficult.
4. **Inability to Learn Automatically:** Unlike modern machine learning systems, traditional expert systems cannot learn from new experiences; rules must be coded manually.
5. **Brittleness:** If rules in the Knowledge Base contain contradictions or gaps, the system may crash or produce highly incorrect recommendations without warning.

Chapter 2

Intelligent Agents

2.1 Section 1: Definition of Agent, Environment, and Percept Sequence (Q2.1)

2.1.1 1. Definitions

- **Agent:** An **Agent** is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- **Environment:** The **Environment** is the surrounding or context in which the agent operates and interacts. It is the world that the agent perceives and acts upon.
- **Percept Sequence:** An agent's **Percept Sequence** is the complete history of everything that the agent has perceived (received via its sensors) from the beginning of its operation to the present moment.

2.1.2 2. Relationship and Core Mathematical Formulation

An agent's behavior is mathematically described by the **Agent Function** (f), which maps any given percept sequence to a specific action:

$$f : P^* \rightarrow A$$

Where:

- P^* is the set of all possible percept sequences (of any length).
- A is the set of all possible actions.

In practice, the agent function is implemented by an **Agent Program**, which runs on a physical architecture (hardware/processor):

$$\text{Agent} = \text{Architecture} + \text{Program}$$

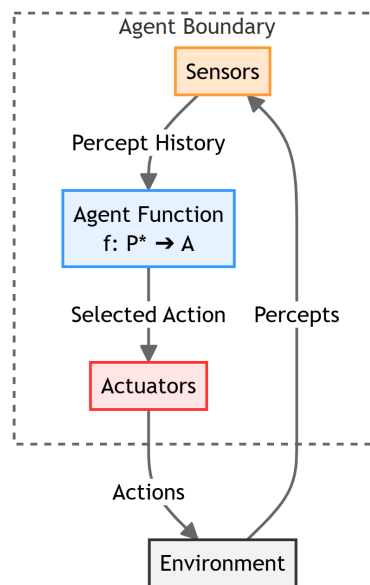


Figure 2.1: Environment-Agent Relationship

2.2 Section 2: PEAS Description (Q2.2)

2.2.1 1. Definition of PEAS

To design a rational agent, we must specify its task environment, which is summarized using the **PEAS** framework:

- **P – Performance Measure:** The metric used to evaluate how successful the agent's behavior is.
- **E – Environment:** The external context, objects, and variables the agent interacts with.
- **A – Actuators:** The devices or output mechanisms the agent uses to perform actions.
- **S – Sensors:** The inputs or devices the agent uses to perceive the environment.

2.2.2 2. Comparative PEAS Examples for Standard Systems

Agent Type	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Automated Taxi Driver	Safe arrival, speed, legal driving, passenger comfort, profits.	Roads, other traffic, pedestrians, weather, customers.	Steering wheel, accelerator, brakes, horn, display/speech.	Cameras, LIDAR, sonar, GPS, speedometer, odometer, engine sensors.
Medical Diagnosis System	Patient health, minimized cost, no legal liability, diagnostic accuracy.	Patient, hospital staff, laboratory test reports.	Screen displays (diagnoses, prescriptions, tests), alerts.	Keyboard/voice input (symptoms, history, test results).
Part-Picking Robot	% of parts in correct bins, speed, safety (no damaged parts).	Conveyor belt, physical parts, bins.	Jointed robotic arm, hand/gripper, motor controller.	Cameras (vision sensors), pressure/force sensors in gripper.
Refinery Controller	Purity, safety, throughput, energy efficiency, cost minimization.	Pipes, chemical reactors, valves, heaters.	Valves, pumps, heaters, alarms, control panels.	Temperature gauges, pressure sensors, flow-rate sensors.

2.3 Section 3: The Five Basic Agent Structures (Q2.3)

All AI agents can be structured into one of five basic architectural types based on their internal reasoning mechanisms:

2.3.1 1. Simple Reflex Agents

Working Principle: Selects actions based only* on the current percept, ignoring the history of percepts. Uses **condition-action rules** (e.g., "IF brake lights in front are on, THEN initiate braking").

- **Limitation:** Only works if the environment is fully observable.
- **Diagram:**

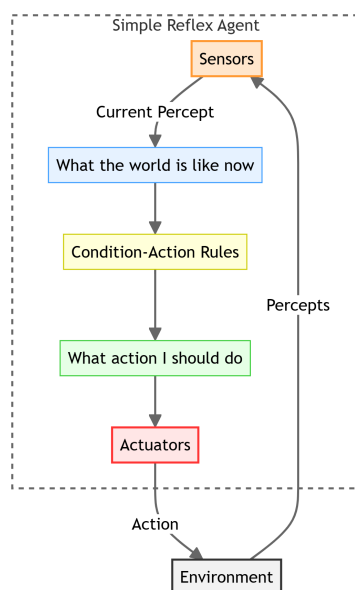


Figure 2.2: Simple Reflex Agent Structure

2.3.2 2. Model-Based Reflex Agents

- **Working Principle:** Maintains an internal **state** (model of the world) to track aspects of the environment that cannot be seen right now. It updates the state based on how the world evolves and what the agent's actions do.
- **Diagram:**

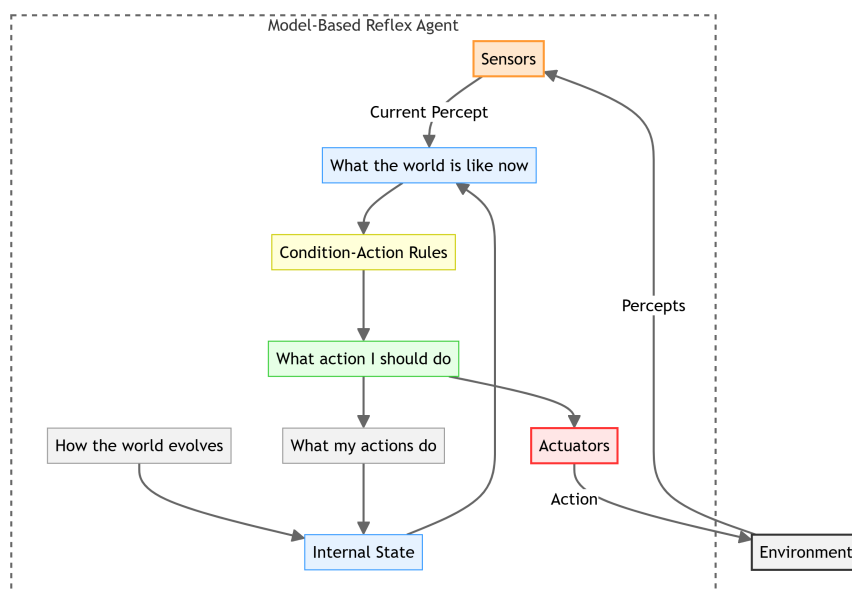


Figure 2.3: Model-Based Reflex Agent Structure

2.3.3 3. Goal-Based Agents

- **Working Principle:** Combines the world model with explicit **goal** information to choose actions that will achieve the desired goal state. Uses search and planning.

- **Diagram:**

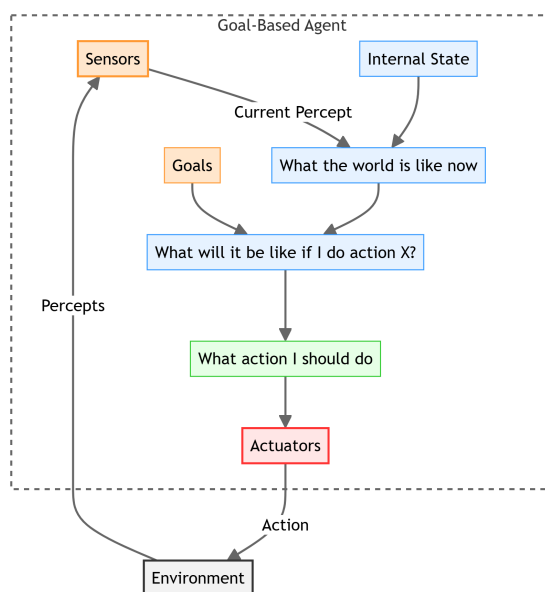


Figure 2.4: Goal-Based Agent Structure

2.3.4 4. Utility-Based Agents

- **Working Principle:** Uses a **utility function** (mapping states to real numbers) to measure how desirable a particular state is (its "happiness" or utility level). Allows trade-offs when goals conflict or have uncertainty.
- **Diagram:**

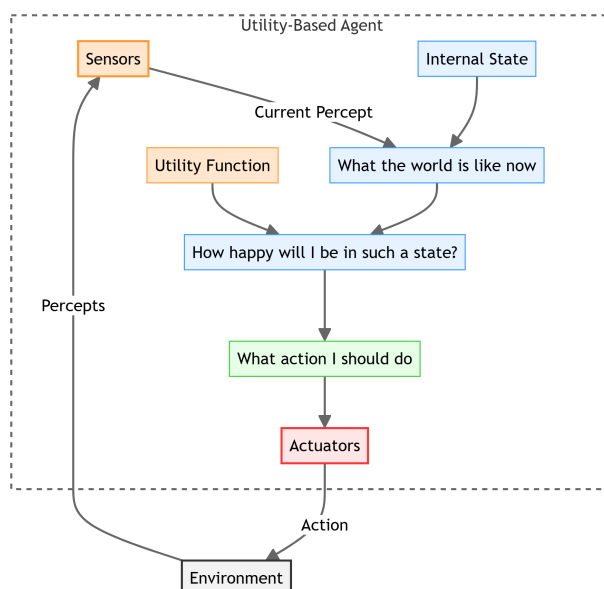


Figure 2.5: Utility-Based Agent Structure

2.3.5 5. Learning Agents

- **Working Principle:** Separates the agent into four conceptual components:

1. **Learning Element:** Responsible for making improvements by learning from experience.
2. **Performance Element:** The standard agent that selects external actions (equivalent to the entire agent in previous structures).
3. **Critic:** Evaluates the agent's behavior against an external performance standard and provides feedback.
4. **Problem Generator:** Suggests new exploratory actions (suboptimal in the short term) to discover new knowledge.

- **Diagram:**

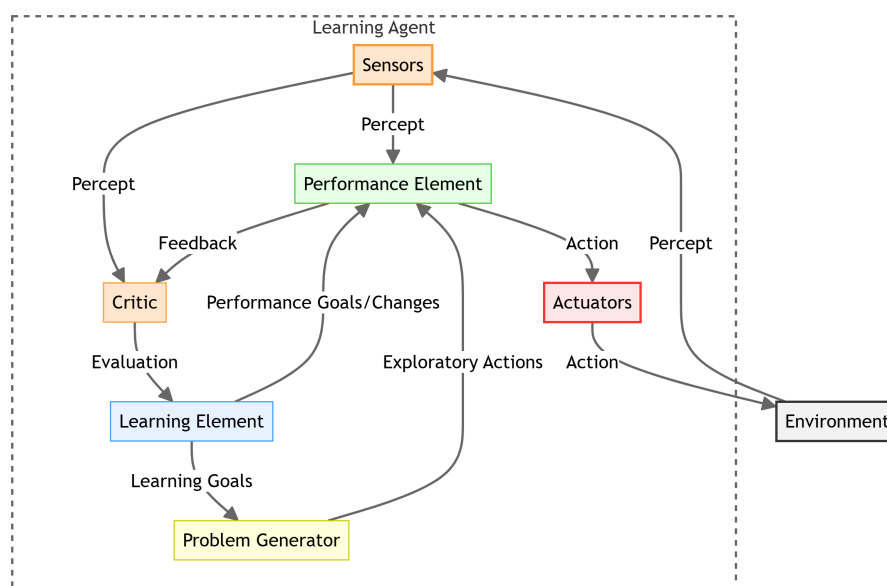


Figure 2.6: Learning Agent Structure

2.4 Section 4: Nature of Environments (Q2.4)

Task environments vary across several properties, which dictate the complexity of the agent design:

2.4.1 1. Environment Properties Explained

- **Fully Observable vs. Partially Observable:**

Fully Observable:* The agent's sensors detect the complete state of the environment at each point in time (e.g., Chess). Partially Observable:* Sensors have noise, limits, or gaps, leaving parts of the state unknown (e.g., Poker, Driving).

- **Deterministic vs. Stochastic:**

Deterministic:* The next state of the environment is completely determined by the current state and the action executed by the agent (e.g., Chess). Stochastic: *The next state is uncertain, containing random probabilities (e.g., weather, coin flips).* Strategic* refers to environments where the uncertainty comes only from other agents.

- **Episodic vs. Sequential:**

Episodic:* The agent's experience is divided into atomic, independent episodes. Action choice in one episode does not affect the next (e.g., part classification on assembly line). Sequential:* Current decisions affect all future decisions and states (e.g., Chess, Driving).

- **Static vs. Dynamic:**

Static:* The environment does not change while the agent is deciding its action (e.g., Crossword puzzle). Dynamic: *The environment continues to change while the agent is formulating its decision (e.g., Driving)*. Semidynamic* means the environment doesn't change, but the agent's performance score decreases with time spent thinking (e.g., Chess with a clock).

- **Discrete vs. Continuous:**

Discrete:* Has a finite or countable number of distinct states, percepts, and actions (e.g., Chess board positions). Continuous:* Variables (speed, location, temperature) are represented by continuous real numbers (e.g., Driving).

- **Single-agent vs. Multi-agent:**

Single-agent:* Only one agent operates in the environment (e.g., Sudoku, Solitaire). Multi-agent:* Multiple interacting agents coexist (e.g., Chess, Traffic). Can be competitive (Chess) or cooperative (Driving rules).

2.4.2 2. Comparison Matrix of Common Environments

Environment	Observable	Deterministic	Episodic	Static	Discrete	Agent
Chess (with clock)	Fully	Strategic	Sequential	Semidynamic	Discrete	Multi
Taxi Driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Image Analysis	Fully	Deterministic	Episodic	Static	Discrete	Single
Medical Diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Sudoku	Fully	Deterministic	Sequential	Static	Discrete	Single

Chapter 3

Solving Problems by Searching (Uninformed Search)

3.1 Section 1: Problem-Solving Agents and Formulation (Q3.1)

3.1.1 1. Definition of Problem-Solving Agents

A **Problem-Solving Agent** is a type of goal-based agent that decides what to do by finding sequences of actions that lead to desirable states. It uses search algorithms to plan ahead before executing actions in the physical world.

3.1.2 2. Steps in Problem Formulation

Problem formulation is the process of deciding what actions and states to consider, given a goal. It consists of 5 core components:

1. **Initial State:** The state in which the agent starts (e.g., $In(Arad)$).
2. **Actions:** The set of possible actions available to the agent at a given state. Mathematically, $Actions(s)$ returns the set of actions that can be executed in state s .
3. **Transition Model:** A description of what each action does. It is represented by a function $Result(s, a)$ that returns the state reached by executing action a in state s .
4. **Goal Test:** A function that determines whether a given state is a goal state. Can be explicit (e.g., $s = In(Bucharest)$) or implicit (e.g., s satisfies a check like checkmate).
5. **Path Cost:** A function that assigns a numeric cost to a path. The step cost of taking action a in state s to reach state s' is denoted by $c(s, a, s')$. The path cost $g(n)$ is the sum of step costs along the path.

3.1.3 3. State Space Representation

The **State Space** is the set of all states reachable from the initial state by any sequence of actions. It is represented as a directed graph where:

- **Nodes:** Represent individual states.
- **Directed Edges:** Represent actions (transitions).

3.2 Section 2: Evaluation Criteria for Search Algorithms (Q3.2)

A search algorithm's performance is evaluated using four standard criteria:

1. **Completeness:** Is the algorithm guaranteed to find a solution if one exists?
2. **Time Complexity:** How long (number of generated nodes or steps) does it take to find a solution?
3. **Space Complexity:** How much memory does the algorithm require to perform the search?
4. **Optimality:** Does the strategy find the path with the lowest path cost among all solutions?

3.2.1 Key Parameters:

- b : Branching factor (maximum number of successors of any node).
- d : Depth of the shallowest goal node.
- m : Maximum depth of the state space (can be ∞).

3.3 Section 3: BFS vs. DFS Comparison (Q3.3)

3.3.1 1. Feature Comparison Table

Feature / Criteria	Breadth-First Search (BFS)	Depth-First Search (DFS)
Search Strategy	Explores level-by-level (expands all nodes at depth d before depth $d+1$).	Explores deep into a branch first; backtracks only when it hits a dead end.
Data Structure	FIFO Queue (First-In, First-Out)	LIFO Stack (Last-In, First-Out) or Recursion
Completeness	Yes (if branching factor b is finite).	No (fails in infinite paths or cyclic state spaces).
Time Complexity	$O(b^d)$ (Exponential)	$O(b^m)$ (Exponential; worse than BFS if $m \gg d$)
Space Complexity	$O(b^d)$ (Stores all frontier nodes in memory).	$O(bm)$ (Linear; extremely memory-efficient).
Optimality	Yes (if all step costs are equal/uniform).	No (returns the first path found, not necessarily the shortest).

3.3.2 2. Why DFS is Not Always Complete

DFS is not complete because it can traverse down an infinite path or get trapped in a cycle in the state space graph. If a cycle or infinite path exists on a branch to the left of the goal node, DFS will continue down that branch forever, never backtracking to explore other branches containing the goal.

3.4 Section 4: IDDFS and UCS Algorithms (Q3.4)

3.4.1 1. Iterative Deepening Depth-First Search (IDDFS)

- **Working Principle:** Combines the benefits of BFS (optimal and complete) and DFS (low memory). It runs Depth-Limited Search (DLS) repeatedly with increasing depth limits ($l = 0, 1, 2, \dots$) until a goal is found.
- **Evaluation:**

Completeness:* Yes (if b is finite). Optimality:* Yes (if step costs are uniform). Time Complexity:* $O(b^d)$ Space Complexity:* $O(bd)$ (Linear space).

IDDFS Algorithm:

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to infinity do
    result <- DEPTH-LIMITED-SEARCH(problem, depth)
    if result != cutoff then return result
```

3.4.2 2. Uniform-Cost Search (UCS)

- **Working Principle:** Generalizes BFS for arbitrary step costs. It expands the frontier node n that has the lowest accumulated path cost $g(n)$ from the start state.
- **Evaluation:**

Completeness:* Yes (if all step costs $\geq \epsilon > 0$). Optimality:* Yes (always expands the lowest-cost path first). Time & Space Complexity:* $O(b^{1+\lceil C^*/\epsilon \rceil})$, where C^* is the cost of the optimal solution and ϵ is the minimum step cost.

UCS Algorithm:

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node <- a node with State = problem.INITIAL-STATE, Path-Cost = 0
  frontier <- a priority queue ordered by Path-Cost, with node as the
    only element
  explored <- an empty set

  loop do
    if IS-EMPTY(frontier) then return failure
    node <- POP(frontier) /* chooses the lowest-cost node */
    if problem.GOAL-TEST(node.State) then return SOLUTION(node)
    add node.State to explored
    for each action in problem.ACTIONS(node.State) do
      child <- CHILD-NODE(problem, node, action)
      if child.State is not in explored or frontier then
        INSERT(child, frontier)
      else if child.State is in frontier with higher Path-Cost then
        replace that frontier node with child
```

3.5 Section 5: Formulations of Classic Puzzles (Q3.5)

3.5.1 1. 8-Puzzle Problem Formulation

- **States:** Board configurations specifying the locations of the 8 tiles (1 to 8) and the blank space in a 3×3 grid.
- **Initial State:** Any valid board configuration (e.g., a scrambled grid).
- **Actions:** Move the blank space 'Left', 'Right', 'Up', or 'Down'.
- **Transition Model:** Returns the board configuration after shifting the blank space in the specified direction.
- **Goal Test:** Matches a target configuration (e.g., tiles in numerical order with blank at the bottom right).
- **Path Cost:** Each step costs 1 ($g(n) = \text{number of moves}$).

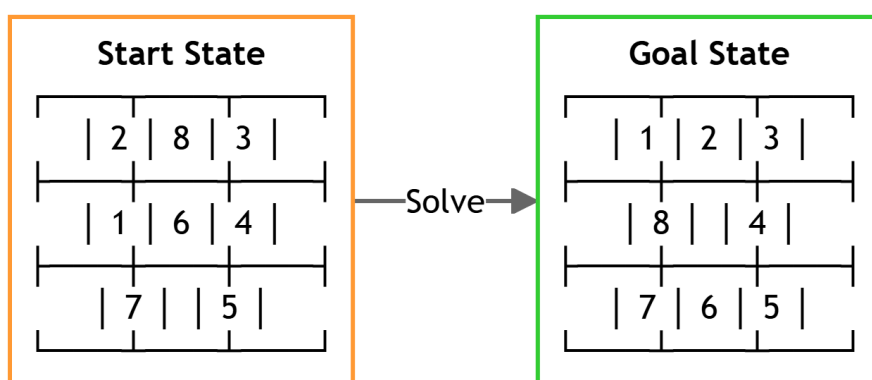


Figure 3.1: 8-Puzzle Start and Goal States

3.5.2 2. The Missionaries and Cannibals Problem (3M, 3C)

- **Problem Statement:** Three missionaries and three cannibals must cross a river using a boat that holds at most two people, with the constraint that the number of cannibals must never exceed the number of missionaries on either bank (otherwise the cannibals eat the missionaries). The boat must have at least one person to cross.

State Formulation:

- Represented as a 3-tuple: (M, C, B)
- M : Number of missionaries on the **left bank** ($0 \leq M \leq 3$).
- C : Number of cannibals on the **left bank** ($0 \leq C \leq 3$).
- B : Location of the boat (1 for left bank, 0 for right bank).

Note:* The state of the right bank is implicitly $(3 - M, 3 - C, 1 - B)$.

Constraints (Safety Conditions):

For any state (M, C, B) , it is safe if and only if:

1. If $M > 0$, then $M \geq C$ (Left bank is safe).
2. If $3 - M > 0$, then $(3 - M) \geq (3 - C)$ (Right bank is safe).

Actions (Transitions):

Let (m, c) be the number of missionaries and cannibals placed in the boat ($1 \leq m + c \leq 2$).

- If $B = 1$ (moving left to right):

$$\text{Result}((M, C, 1), (m, c)) = (M - m, C - c, 0)$$

- If $B = 0$ (moving right to left):

$$\text{Result}((M, C, 0), (m, c)) = (M + m, C + c, 1)$$

Step-by-Step Solution Path:

Starting at $(3, 3, 1)$ and ending at $(0, 0, 0)$:

Step	Current State (M, C, B)	Action Taken (Boat Contents)	Direction	Resulting State
0	(3, 3, 1)	Send 2 Cannibals (0M, 2C)	Left $\rightarrow$ Right	(3, 1, 0)
1	(3, 1, 0)	Return 1 Cannibal (0M, 1C)	Right $\rightarrow$ Left	(3, 2, 1)
2	(3, 2, 1)	Send 2 Cannibals (0M, 2C)	Left $\rightarrow$ Right	(3, 0, 0)
3	(3, 0, 0)	Return 1 Cannibal (0M, 1C)	Right $\rightarrow$ Left	(3, 1, 1)
4	(3, 1, 1)	Send 2 Missionaries (2M, 0C)	Left $\rightarrow$ Right	(1, 1, 0)
5	(1, 1, 0)	Return 1M & 1C (1M, 1C)	Right $\rightarrow$ Left	(2, 2, 1)
6	(2, 2, 1)	Send 2 Missionaries (2M, 0C)	Left $\rightarrow$ Right	(0, 2, 0)
7	(0, 2, 0)	Return 1 Cannibal (0M, 1C)	Right $\rightarrow$ Left	(0, 3, 1)
8	(0, 3, 1)	Send 2 Cannibals (0M, 2C)	Left $\rightarrow$ Right	(0, 1, 0)
9	(0, 1, 0)	Return 1 Cannibal (0M, 1C)	Right $\rightarrow$ Left	(0, 2, 1)
10	(0, 2, 1)	Send 2 Cannibals (0M, 2C)	Left $\rightarrow$ Right	(0, 0, 0)

3.5.3 3. The Water Jug Problem (4-Gal and 3-Gal Jugs)

- **Problem Statement:** You have a 4-gallon jug and a 3-gallon jug with no measuring marks. You need to get exactly 2 gallons of water into the 4-gallon jug using a pump.

State Formulation:

Represented as a 2-tuple: (x, y)

- x : Gallons of water in the 4-gallon jug ($0 \leq x \leq 4$).
- y : Gallons of water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial State:** $(0, 0)$
- **Goal State:** $(2, y)$ where y can be any value.

Production Rules:

Rule Number	Condition	Action	Result	Description
1	$x < 4$	Fill 4-Gal Jug	$(4, y)$	Fill the 4-gallon jug completely.
2	$y < 3$	Fill 3-Gal Jug	$(x, 3)$	Fill the 3-gallon jug completely.
3	$x > 0$	Empty 4-Gal Jug	$(0, y)$	Pour all water out of the 4-gallon jug.
4	$y > 0$	Empty 3-Gal Jug	$(x, 0)$	Pour all water out of the 3-gallon jug.
5	$x + y \geq 4$ and $y > 0$	Pour 3-Gal $\rightarrow$ 4-Gal until full	$(4, y - (4 - x))$	Fill 4-Gal jug from 3-Gal jug.
6	$x + y \geq 3$ and $x > 0$	Pour 4-Gal $\rightarrow$ 3-Gal until full	$(x - (3 - y), 3)$	Fill 3-Gal jug from 4-Gal jug.
7	$x + y \leq 4$ and $y > 0$	Pour all 3-Gal $\rightarrow$ 4-Gal	$(x + y, 0)$	Empty 3-Gal jug contents into 4-Gal.
8	$x + y \leq 3$ and $x > 0$	Pour all 4-Gal $\rightarrow$ 3-Gal	$(0, x + y)$	Empty 4-Gal jug contents into 3-Gal.

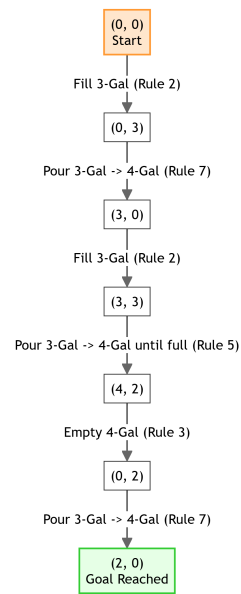
Traced Solution Steps:

Figure 3.2: Water Jug Traced Solution Path

Chapter 4

Informed Search & Exploration

4.1 Section 1: Greedy Best-First Search vs. A* Search (Q4.1)

4.1.1 1. Comparison Table

Feature / Criteria	Greedy Best-First Search	A* Search
Evaluation Function	$f(n) = h(n)$ (Uses only the heuristic estimate to goal).	$f(n) = g(n) + h(n)$ (Combines cost-so-far and estimated cost-to-goal).
Search Bias	Highly greedy. Can easily wander down false paths if heuristics are inaccurate.	Balanced. Optimizes both total path cost and remaining distance.
Completeness	No (can get stuck in loops in cyclic graphs).	Yes (if branching factor b is finite and step costs $c \geq \epsilon > 0$).
Optimality	No (returns the first path found, which is often suboptimal).	Yes (if $h(n)$ is admissible for trees, or consistent for graphs).
Space Complexity	$O(b^m)$ (Frontier nodes are kept in memory).	$O(b^d)$ (Stores all generated nodes in memory; high memory cost).

4.1.2 2. Conditions for A* Optimality and Completeness

A* Search is guaranteed to be:

1. **Optimal (Tree Search):** If the heuristic $h(n)$ is **admissible** (never overestimates the true cost to reach the goal).
2. **Optimal (Graph Search):** If the heuristic $h(n)$ is **consistent (or monotonic)** (satisfies triangle inequality).
3. **Complete:** If the branching factor b is finite, and all step costs are positive ($c(s, a, s') \geq \epsilon > 0$).

4.2 Section 2: Admissibility and Consistency of Heuristics (Q4.2)

4.2.1 1. Admissibility

A heuristic function $h(n)$ is **admissible** if it never overestimates the actual cost to reach the goal state from node n . Mathematically:

$$0 \leq h(n) \leq h^*(n)$$

Where $h^*(n)$ is the true, optimal cost to reach the goal from n . (Note: $h(Goal) = 0$).

4.2.2 2. Consistency (Monotonicity)

A heuristic $h(n)$ is **consistent** (or monotonic) if, for every node n and every successor n' generated by action a , the estimated cost of reaching the goal from n is no greater than the step cost of moving from n to n' plus the estimated cost of reaching the goal from n' :

$$h(n) \leq c(n, a, n') + h(n')$$

This is equivalent to the **triangle inequality** in geometry.

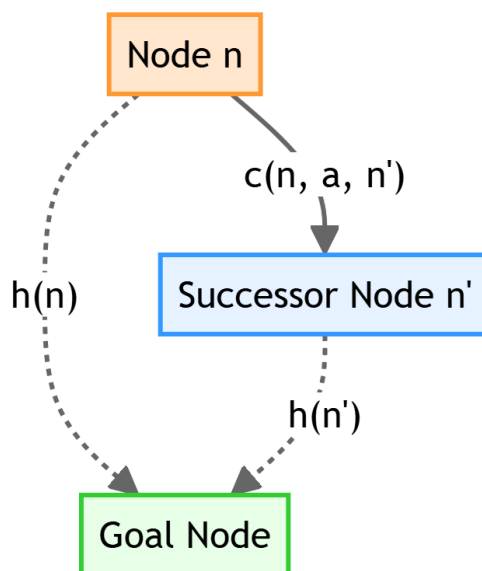


Figure 4.1: Heuristic Consistency Inequality Triangle

4.2.3 3. Proof: Consistent Heuristic is Admissible

We can prove this by induction on the number of steps k from node n to the goal.

- **Base Case:** If $k = 0$, n is the goal. $h(Goal) = 0 \leq h^*(Goal) = 0$. (True).
- **Inductive Step:** Suppose $h(n_i) \leq h^*(n_i)$ holds for all nodes n_i that are k steps from the goal. Let n be $k + 1$ steps from the goal. By consistency:

$$h(n) \leq c(n, a, n') + h(n')$$

Since n' is k steps from the goal, by inductive hypothesis, $h(n') \leq h^*(n')$. Therefore:

$$h(n) \leq c(n, a, n') + h^*(n')$$

Since $c(n, a, n') + h^*(n')$ is the optimal cost $h^*(n)$ from n via n' , we get:

$$h(n) \leq h^*(n)$$

Thus, any consistent heuristic is admissible. ■

4.3 Section 3: Simulated Annealing Search (Q4.3)

4.3.1 1. Working Principle

Simulated Annealing is a local search technique based on the metallurgical process of annealing (heating and slowly cooling metal to reduce defects). It avoids getting stuck in local maxima by occasionally accepting "bad" moves (moves that decrease the value of the evaluation function) with a probability that decreases over time.

4.3.2 2. Algorithm Steps

1. Initialize the current state and set the temperature T to a high starting value.
2. At each step, select a random neighbor of the current state.
3. Calculate the value difference: $\Delta E = \text{Value}(\text{Neighbor}) - \text{Value}(\text{Current})$.
4. If $\Delta E > 0$ (the neighbor is better), accept the move immediately.
5. If $\Delta E \leq 0$ (the neighbor is worse), accept the move with probability:

$$P = e^{\frac{\Delta E}{T}}$$

1. Decrease the temperature T according to a **cooling schedule**.
2. Repeat until $T \approx 0$ or a termination condition is met.

4.3.3 3. Impact of the Cooling Schedule

- **If the temperature is lowered slowly enough:** The probability of accepting worse states decreases gradually. The algorithm behaves like a random walk initially (exploring widely) and transitions smoothly into a pure greedy local search (hill-climbing) at the end. This is mathematically guaranteed to converge to the **global optimum** with probability 1.
- **If the cooling schedule is too fast:** The algorithm freezes too quickly, trapping the system in a **local optimum** (suboptimal state).

4.4 Section 4: Mathematical Proofs in Informed Search (Q4.4)

4.4.1 1. Proof: A* Tree Search is Admissible (Optimal)

We prove A* optimality by contradiction. Let G_2 be a suboptimal goal node (i.e., $g(G_2) > C^*$, where C^* is the optimal path cost). Let G be the optimal goal node ($g(G) = C^*$). Suppose A* terminates by selecting G_2 from the priority queue.

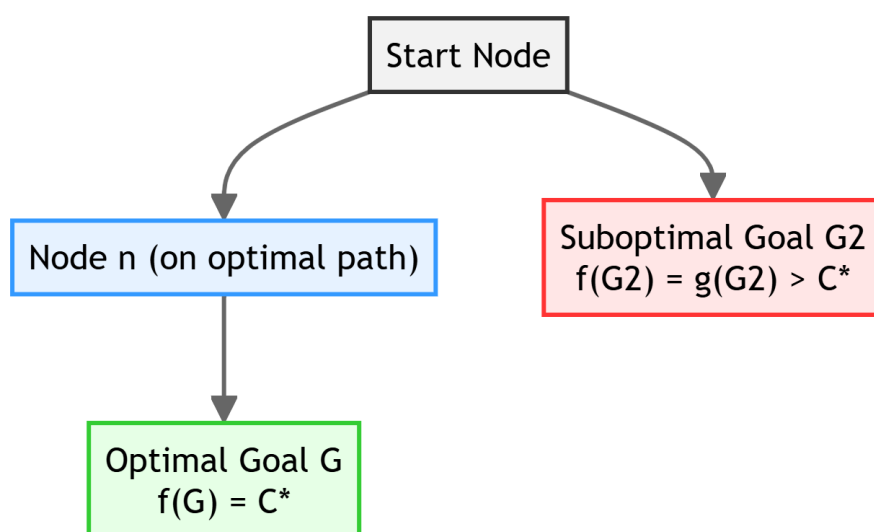


Figure 4.2: A* Optimality Contradiction Tree

At some point before selecting G_2 , there must be a node n on the frontier that lies on the optimal path to G .

- Since $h(n)$ is admissible: $h(n) \leq h^*(n)$.
- Therefore:

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = f^*(n) = C^*$$

- Since G_2 is a goal node, $h(G_2) = 0$, which gives:

$$f(G_2) = g(G_2) + h(G_2) = g(G_2) > C^*$$

- Combining the two inequalities:

$$f(n) \leq C^* < f(G_2)$$

Since $f(n) < f(G_2)$, the priority queue would select and expand node n before selecting G_2 . This contradicts the assumption that G_2 was popped first. Thus, A must select the optimal goal node. ■

4.4.2 2. Proof: Consistent Heuristic implies Monotonic f-values

Let n' be a successor of node n generated by action a . The f -value of n is:

$$f(n) = g(n) + h(n)$$

The f -value of n' is:

$$f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n')$$

By consistency of the heuristic:

$$h(n) \leq c(n, a, n') + h(n')$$

Adding $g(n)$ to both sides:

$$g(n) + h(n) \leq g(n) + c(n, a, n') + h(n')$$

Which simplifies directly to:

$$f(n) \leq f(n')$$

Thus, the f -value of nodes along any path is monotonically non-decreasing when using a consistent heuristic. ■

4.5 Section 5: A* Search Solving the 8-Puzzle (Q4.5)

4.5.1 1. Heuristic Selection

We use the **Manhattan Distance** (h_{MD}) as our admissible and consistent heuristic. It is the sum of the horizontal and vertical distances of the tiles from their goal positions.

$$h_{MD} = \sum_{i=1}^8 (|x_{current,i} - x_{goal,i}| + |y_{current,i} - y_{goal,i}|)$$

4.5.2 2. Trace Example: Initial State to Goal State

Let our Initial State and Goal State be:

Initial State (S)		Goal State (G)
+---+---+---+		+---+---+---+
2 8 3		1 2 3
+---+---+---+		+---+---+---+
1 6 4	---->	8 4
+---+---+---+		+---+---+---+
7 5		7 6 5
+---+---+---+		+---+---+---+

Step-by-Step Search Tree Expansion:

1. Root Node (S): $g(S) = 0$

- Manhattan distances:

- '1': $(1, 0) \rightarrow (0, 0) \Rightarrow \text{dist} = 1$

- '2': $(0, 0) \rightarrow (1, 0) \Rightarrow \text{dist} = 1$

- '3': $(2, 0) \rightarrow (2, 0) \Rightarrow \text{dist} = 0$

- '4': $(2, 1) \rightarrow (2, 1) \Rightarrow \text{dist} = 0$

- '5': $(2, 2) \rightarrow (2, 2) \Rightarrow \text{dist} = 0$

- '6': $(1, 1) \rightarrow (1, 2) \Rightarrow \text{dist} = 1$

- '7': $(0, 2) \rightarrow (0, 2) \Rightarrow \text{dist} = 0$

- '8': $(1, 0) \rightarrow (0, 1) \Rightarrow \text{dist} = 2$

- $h(S) = 1 + 1 + 0 + 0 + 0 + 1 + 0 + 2 = 5$

- $f(S) = 0 + 5 = 5$. Expand S (blank can move Left, Right, or Up).

1. Successors of S:

- **Node A (Move Left):** Blank shifts to $(0, 2)$ (swaps with '7').

- $g(A) = 1, h(A) = 6 \Rightarrow f(A) = 7$.

- **Node B (Move Right):** Blank shifts to $(2, 2)$ (swaps with '5').

- $g(B) = 1, h(B) = 6 \Rightarrow f(B) = 7$.

- **Node C (Move Up):** Blank shifts to $(1, 1)$ (swaps with '6').

- $g(C) = 1, h(C) = 4$ (tiles '6' and '8' move closer) $\Rightarrow f(C) = 1 + 4 = 5$.

Decision:* Pop and expand **Node C** ($f(C) = 5$).

1. Successors of Node C: (Blank at $(1, 1)$ can move Left, Right, Up, or Down).

- **Node C1 (Move Left):** Swaps with '1'. $g(C1) = 2, h(C1) = 3 \Rightarrow f(C1) = 5$.

- **Node C2 (Move Right):** Swaps with '4'. $g(C2) = 2, h(C2) = 5 \Rightarrow f(C2) = 7$.

- **Node C3 (Move Up):** Swaps with '8'. $g(C3) = 2, h(C3) = 3 \Rightarrow f(C3) = 5$.
- **Node C4 (Move Down):** Swaps with '6' (reverts to start S). $g(C4) = 2, h(C4) = 5 \Rightarrow f(C4) = 7$.

Decision:* Node C1 and Node C3 both have $f = 5$. Let's expand **Node C1**.

1. **Successors of Node C1:** (Blank at (0, 1) can move Up, Down, or Right).

- **Node C1_1 (Move Up):** Swaps with '2'.
- Board configuration is: '[, 2, 3] / [1, 8, 4] / [7, 6, 5]'.
- $g(C1_1) = 3, h(C1_1) = 2 \Rightarrow f(C1_1) = 5$.

Decision:* Pop and expand **Node C1_1** ($f = 5$).

1. **Successors of Node C1_1:** (Blank at (0, 0) can move Right or Down).

- **Node C1_1_1 (Move Right):** Swaps with '2'.
- Board is: '[2, , 3] / [1, 8, 4] / [7, 6, 5]'. (Repeated state, skip).
- **Node C1_1_2 (Move Down):** Swaps with '1'.
- Board is: '[1, 2, 3] / [, 8, 4] / [7, 6, 5]'.
- $g = 4, h = 1$ (only '8' is out of place) $\Rightarrow f = 5$.

Decision:* Pop and expand **Node C1_1_2** ($f = 5$).

1. **Successors of Node C1_1_2:** (Blank at (0, 1) can move Up, Down, or Right).

- **Node C1_1_2_1 (Move Right):** Swaps with '8'.
- Board is: '[1, 2, 3] / [8, , 4] / [7, 6, 5]'.
- $g = 5, h = 0$ (Goal state reached!) $\Rightarrow f = 5$.

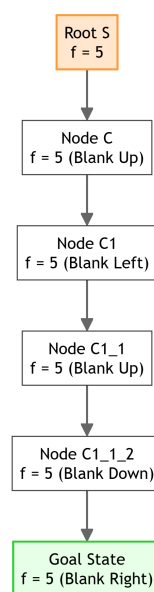


Figure 4.3: 8-Puzzle State Transition Trace

4.6 Section 6: Local Search and Optimization Techniques (Q4.6)

4.6.1 1. Hill-Climbing Search

- **Definition:** A local search algorithm that continually moves in the direction of increasing value (upland) to find the peak. It does not maintain a search tree, only the current state.
- **Three Major Limitations:**
 1. **Local Maxima:** A peak that is higher than each of its neighboring states, but lower than the global maximum. The algorithm gets stuck here.
 2. **Ridges:** A sequence of local maxima forming a ridge. The slope leads up the ridge, but neighbors in any cardinal direction lead down, causing the algorithm to oscillate and stall.
 3. **Plateaus:** A flat area of the state space where all neighboring states have the same value (flat local maximum or shoulder). The search gets lost in a random walk.
- **Solutions:**

Stochastic Hill Climbing:* Chooses randomly from among the uphill moves. Random-Restart Hill Climbing:* Conducts a series of searches from randomly generated initial states, saving the best peak found.

4.6.2 2. Local Beam Search

- **Working Principle:** Instead of keeping track of one state (like Hill-Climbing), Local Beam Search tracks k states. It begins with k randomly generated states. At each step, all the successors of all k states are generated. If any one is a goal, the algorithm halts. Otherwise, it selects the k best successors from the complete list and repeats.
- **Difference from Multi-Start Hill-Climbing:** In random-restart hill-climbing, each search runs independently. In local beam search, information is shared among the parallel threads: useful steps are highlighted, and poor performing paths are abandoned in favor of promising ones.

4.6.3 3. AO* Search (AND-OR Graph Search)

Definition: AO is an informed search algorithm used to find solutions in **AND-OR graphs**, which represent problems that can be decomposed into smaller sub-problems. OR node:* Represents choices (e.g., to solve problem P, solve either P1 or P2). AND node: *Represents decomposition (e.g., to solve problem P, solve both* P3 and P4).*

- **Example Scenario:**

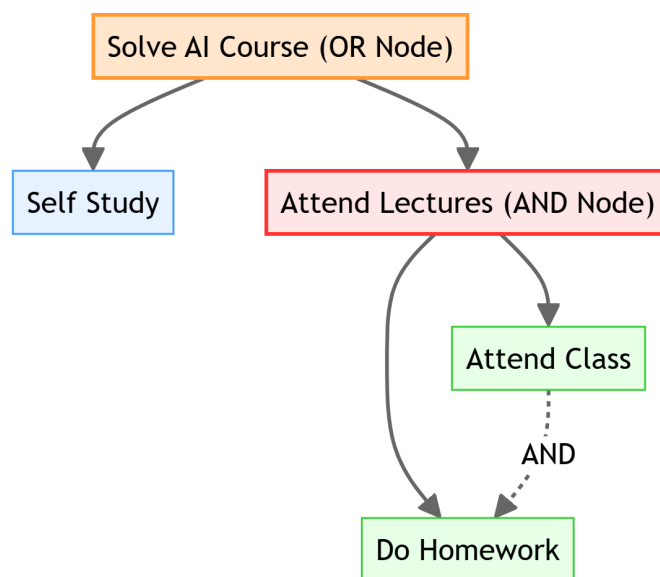


Figure 4.4: AND-OR Graph Decompositions

To clear the course via lectures, the agent must complete *both* sub-tasks (AND branch). AO* computes the cost using:

$$f(n) = g(n) + \sum h(\text{sub-problems})$$

It updates costs backward from child nodes to parent nodes to find the optimal sub-tree.

4.7 Section 7: Informed Search Properties & Linear Space Search (Q4.7) [5M][[]]

Standard heuristic searches (like A* and Greedy Best-First Search) are powerful but suffer from high memory requirements. Understanding their complexity limits and alternative linear-memory algorithms is essential for practical pathfinding.

4.7.1 1. Heuristic Search Concept & Greedy Expansion

- **Concept:** Heuristic search uses problem-specific knowledge (beyond the basic problem definition) to guide the search. While uninformed search (BFS/DFS) blindly traverses the tree, informed search uses a **heuristic function** $h(n)$ to estimate the cost of the cheapest path from node n to the goal.
- **Node Expansion Criteria: Greedy Best-First Search (GBFS)** expands the node that is estimated to be **closest to the goal** (the node with the lowest $h(n)$ value, setting its evaluation function $f(n) = h(n)$).

4.7.2 2. Space Complexity of Greedy Search

Frontier Storage: Like A, Greedy Search maintains a frontier (priority queue) of all generated but unexpanded nodes.

- **Worst-case Complexity:** In the worst case, the algorithm must store all nodes along the paths in memory. This leads to an exponential space complexity of $O(b^l)$ (or $O(b^m)$), where b is the branching factor and l is the maximum depth of the search space.

4.7.3 3. Recursive Best-First Search (RBFS)

Recursive Best-First Search (RBFS) is an informed search algorithm that mimics the best-first behavior of A* search but runs in **linear space**.

A. Working Principle:

1. RBFS uses a recursive depth-first execution style.
2. Instead of keeping all nodes in memory, it only keeps track of the current path and its immediate siblings.
3. **The f -limit:** Each recursive call is passed an **f -limit** representing the f -value of the best alternative path available from any ancestor of the current node.
4. **Back-propagation & Backtracking:**
 - If the current node's f -value exceeds the f -limit, the recursion backtracks (unwinds) to explore the alternative path.

Crucially:* As the recursion unwinds, RBFS replaces the f -value of each ancestor node in the path with the **best f -value of its children**. Memory/Recall Trade-off:* This allows the algorithm to "remember" the quality of the best leaf in the pruned subtree, so it can decide whether to re-explore it later without having to keep the entire subtree structure in RAM.

B. Complexity Analysis:

Space Complexity: $O(bd)$ (where b is the branching factor and d is the search depth). Since it only keeps the current path and siblings in memory, its space usage is linear, resolving A's major memory bottleneck.

- **Time Complexity:** In the worst case, RBFS can have exponential time complexity ($O(b^d)$) because it is prone to **node regeneration**—it may repeatedly expand and throw away the same subtrees if the f -values fluctuate frequently.
- **Optimality:** RBFS is optimal if the heuristic function $h(n)$ is **admissible**.

Chapter 5

Constraint Satisfaction Problems

5.1 Section 1: Introduction to Constraint Satisfaction Problems (Q5.1)

5.1.1 1. Definition of a CSP

A **Constraint Satisfaction Problem (CSP)** is a mathematical problem defined as a set of objects whose state must satisfy a number of constraints or limitations. Unlike standard search problems where states are "black boxes", CSP states use a structured representation defined by variables and constraints.

A CSP is formally defined by three components:

1. **Variables (V):** A set of variables, $\{V_1, V_2, \dots, V_n\}$.
2. **Domains (D):** A set of domains, $\{D_1, D_2, \dots, D_n\}$, where each domain D_i is a set of allowable values for variable V_i .
3. **Constraints (C):** A set of constraints that specify allowable combinations of values. Each constraint consists of a tuple of variables and the relations between them (e.g., $V_1 \neq V_2$).

5.1.2 2. Map-Coloring Example

- **Problem:** Color the map of Australia (with territories: Western Australia WA , Northern Territory NT , Queensland Q , New South Wales NSW , Victoria V , South Australia SA , Tasmania T) using three colors ('Red', 'Green', 'Blue') such that no two adjacent territories share the same color.

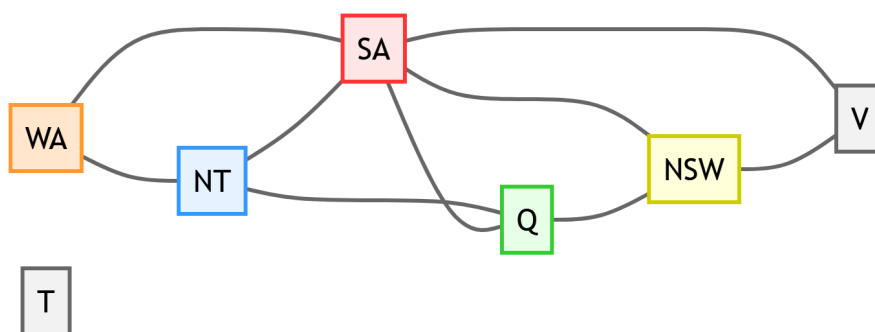


Figure 5.1: Australia Map-Coloring Outline

- **Variables (V):** $\{WA, NT, Q, NSW, V, SA, T\}$

- **Domains (D):** $D_i = \{\text{Red, Green, Blue}\}$ for all variables.
- **Constraints (C):** Adjacent regions must have different colors:
- $WA \neq NT, WA \neq SA$
- $NT \neq SA, NT \neq Q$
- $SA \neq Q, SA \neq NSW, SA \neq V$
- $Q \neq NSW$
- $NSW \neq V$
- **Solution Example:**
- $WA = \text{Red}, NT = \text{Green}, Q = \text{Red}, NSW = \text{Green}, V = \text{Red}, SA = \text{Blue}, T = \text{Red}.$

5.2 Section 2: Solving Cryptarithmic Problems (Q5.2)

5.2.1 1. Problem Formulation

Cryptarithmic is a constraint satisfaction puzzle where letters in a mathematical sum represent unique digits from 0 to 9.

```

S E N D
+ M O R E
-----
M O N E Y

```

Variables (V):

- Core letter variables: $\{S, E, N, D, M, O, R, Y\}$
- Auxiliary carry variables: $\{C_1, C_2, C_3, C_4\}$ where $C_i \in \{0, 1\}$ represents the carry from the i -th column from right to left.

Domains (D):

- For letters S and M (leading digits, cannot be 0): $D_S, D_M = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- For other letters: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- For carries: $\{0, 1\}$

Constraints (C):

1. **Global Alldifferent Constraint:** $Alldifferent(S, E, N, D, M, O, R, Y)$ (each letter must be assigned a unique digit).
2. **Column Addition Constraints:**

Column 1 (Units):* $D + E = Y + 10 \times C_1$ Column 2 (Tens):* $C_1 + N + R = E + 10 \times C_2$ Column 3 (Hundreds):* $C_2 + E + O = N + 10 \times C_3$ Column 4 (Thousands):* $C_3 + S + M = O + 10 \times C_4$ Column 5 (Ten-Thousands):* $C_4 = M$

5.2.2 2. Step-by-Step Deduction Process

1. Determine M :

- From Column 5: $C_4 = M$.
- Since C_4 is a carry from Column 4, and the maximum possible sum of two single digits plus a carry is $9 + 9 + 1 = 19$, the carry C_4 can only be 1.
- Therefore: $M = 1$, and $C_4 = 1$.

1. Analyze Column 4:

- $C_3 + S + M = O + 10 \times C_4 \Rightarrow C_3 + S + 1 = O + 10$.
- Since $S \leq 9$ and $C_3 \leq 1$, the maximum left-hand side is $1 + 9 + 1 = 11$.
- Therefore, O must be 0 or 1.
- Since $M = 1$ and all letters must be unique, O cannot be 1.
- Therefore: $O = 0$.
- Substituting $O = 0$: $C_3 + S + 1 = 10 \Rightarrow S = 9 - C_3$.

1. Analyze Column 3:

- $C_2 + E + O = N + 10 \times C_3 \Rightarrow C_2 + E + 0 = N + 10 \times C_3 \Rightarrow C_2 + E = N + 10 \times C_3$.
- If $C_3 = 1$, then $C_2 + E = N + 10$. For this to hold with $E, N \leq 9$, we need $E \geq 9$. But if $E = 9$, then N would equal $C_2 - 1$, which is either -1 or 0 . But $O = 0$, so N cannot be 0 .
- If $E = 8$ and $C_2 = 1$, then $N = 9 - 10 = -1$ (impossible).
- Thus, C_3 cannot be 1.
- Therefore: $C_3 = 0$.
- Since $S = 9 - C_3$, we get: $S = 9$.
- Since $C_3 = 0$, the Column 3 equation becomes: $C_2 + E = N$.
- Since $E \neq N$ (alldifferent), the carry C_2 must be 1.
- Therefore: $C_2 = 1$, which gives: $N = E + 1$.

1. Analyze Column 2:

- $C_1 + N + R = E + 10 \times C_2 \Rightarrow C_1 + N + R = E + 10$.
- Substitute $N = E + 1$ into the equation:

$$C_1 + (E + 1) + R = E + 10 \Rightarrow C_1 + R + 1 = 10 \Rightarrow R = 9 - C_1 - 1 = 8 - C_1$$

- If $C_1 = 0$, then $R = 8$.
- If $C_1 = 1$, then $R = 7$.

1. Analyze Column 1 & Solve:

- We have remaining unassigned digits: $\{2, 3, 4, 5, 6, 7, 8\}$ (since $0, 1, 9$ are assigned).
- Let's test $R = 8$ (implies $C_1 = 0$):
- Then $D + E = Y$ (since $C_1 = 0$).
- We need $D + E \leq 8$ (so no carry is generated).
- Since $N = E + 1$, if $E = 5$, then $N = 6$ (valid). If $E = 6$, $N = 7$.
- Let's test $E = 5$:
- Then $N = 6$.
- Remaining digits: $\{2, 3, 4, 7\}$. (Since $R = 8$, $E = 5$, $N = 6$, $M = 1$, $O = 0$, $S = 9$).
- From Column 1: $D + E = Y \Rightarrow D + 5 = Y$.
- If we choose $D = 7$ from the remaining digits, $7 + 5 = 12 \Rightarrow Y = 2$, and carry $C_1 = 1$.
- But we assumed $C_1 = 0$ (so $R = 8$), which is a contradiction (C_1 cannot be both 0 and 1).
- Therefore, C_1 must be 1, which gives: $R = 7$ and $C_1 = 1$.
- Substitute $C_1 = 1$:
- From Column 1: $D + E = Y + 10$.
- Remaining digits: $\{2, 3, 4, 5, 6, 8\}$ (since $0, 1, 7, 9$ are assigned).
- Since $N = E + 1$, and $R = 7$ is assigned:
- If $E = 5$, then $N = 6$. (Available digits left: $\{2, 3, 4, 8\}$).
- Then $D + 5 = Y + 10 \Rightarrow D - 5 = Y$.
- From available digits $\{2, 3, 4, 8\}$, if we choose $D = 8$, then $Y = 8 - 5 = 3$ (which is available).
- This satisfies all alldifferent constraints!

5.2.3 3. Final Solution Values

- $S = 9$
- $E = 5$
- $N = 6$
- $D = 8$
- $M = 1$
- $O = 0$
- $R = 7$
- $Y = 3$

- Prolog has built-in support for backtracking, logic variables, and unification.
- It features libraries like **CLP(FD)** (Constraint Logic Programming over Finite Domains) which allow developers to declare variables, domain bounds, and constraints mathematically. The language's internal execution engine automatically solves the CSP without the developer writing the search loop manually.
- General-purpose languages (like C, Fortran, or C#) require custom search libraries or engines to achieve the same declarative logic.

Chapter 6

Adversarial Search

6.1 Section 1: Minimax Game-Playing Procedure (Q6.1)

6.1.1 1. Definition and Working Principle

Minimax is a recursive decision-making algorithm used in two-player, zero-sum, perfect-information games (e.g., Chess, Tic-Tac-Toe).

- **Zero-sum:** The utility gained by one player is exactly equal to the utility lost by the opponent.
- **Two Players:**
- **MAX:** Tries to maximize the final utility score.
- **MIN:** Tries to minimize MAX's final utility score (maximizing their own advantage).

The algorithm performs a depth-first search of the game tree. It computes the utility values of leaf nodes and propagates them upward: *At a **MAX node**, the value is the maximum* of the children's values.* *At a **MIN node**, the value is the minimum* of the children's values.*

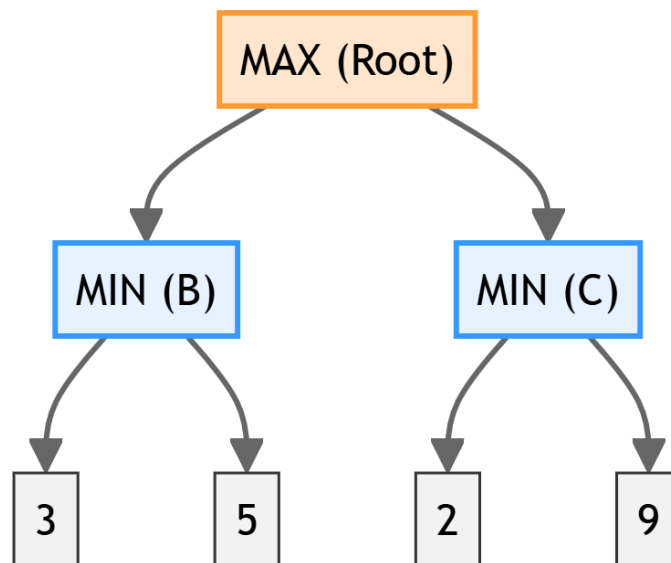


Figure 6.1: Minimax Game Tree

6.1.2 2. Time and Space Complexity

- **Time Complexity:** $O(b^d)$, where b is the branching factor (legal moves) and d is the maximum depth of the game tree.
- **Space Complexity:** $O(bd)$ (Linear space, similar to DFS, storing the active path from root to leaf).

6.2 Section 2: Alpha-Beta Pruning (Q6.2)

6.2.1 1. Working Principle

Alpha-Beta Pruning is an optimization technique for the minimax algorithm that cuts off branches that cannot possibly influence the final decision. It maintains two values throughout the search:

- **Alpha (α):** The value of the best (highest-value) choice found so far at any choice point along the path for MAX. Initialized to $-\infty$.
- **Beta (β):** The value of the best (lowest-value) choice found so far at any choice point along the path for MIN. Initialized to $+\infty$.

The Pruning Rule:

During search, if a node's beta value is less than or equal to its alpha value ($\beta \leq \alpha$), the remaining children of that node do not need to be evaluated (they are pruned).

- **Alpha-Cutoff (α -cutoff):** Occurs at a MIN node when it discovers a value less than or equal to the alpha value of a parent MAX node.
- **Beta-Cutoff (β -cutoff):** Occurs at a MAX node when it discovers a value greater than or equal to the beta value of a parent MIN node.

6.2.2 2. Trace Example: 3-Level Branching Game Tree

Let's execute Minimax and Alpha-Beta Pruning on the game tree shown in Figure 1, where **MAX** is at the root node (A), **MIN** is at level 1 (B, C, D), and **MAX** is at level 2 (E to K).

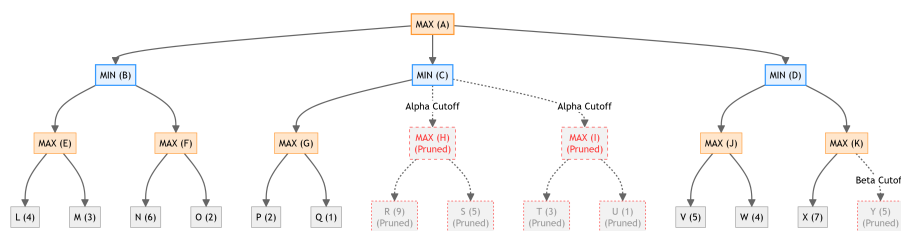


Figure 6.2: Alpha-Beta Pruning Game Tree

Step-by-Step Pruning and Minimax Trace:

Branch B (Left Side of Root):

1. Analyze Node E:

- Children: L (4) and M (3).
- Since E is a MAX node, $Value(E) = \max(4, 3) = 4$.

- Propagate to B: Since B is a MIN node, B's current beta (β_B) becomes $\min(+\infty, 4) = 4$.

1. Analyze Node F:

- Children: N (6) and O (2).
- Since F is a MAX node, $Value(F) = \max(6, 2) = 6$.
- Propagate to B: $\beta_B = \min(4, 6) = 4$.

1. Evaluate Node B:

- Since E (4) and F (6) are evaluated, $Value(B) = \min(4, 6) = 4$.
- Propagate to Root A: Since A is a MAX node, A's alpha (α_A) becomes $\max(-\infty, 4) = 4$.

Branch C (Middle of Root): We search with $\alpha = 4$ and $\beta = +\infty$.

1. Analyze Node G:

- Children: P (2) and Q (1).
- Since G is a MAX node, $Value(G) = \max(2, 1) = 2$.
- Propagate to C (MIN node): β_C becomes $\min(+\infty, 2) = 2$.
- **Check Pruning Condition at C:**
- We have $\alpha_C = 4$ (passed down from A) and $\beta_C = 2$.
- Since $\beta_C \leq \alpha_C$ ($2 \leq 4$), an **Alpha-Cutoff** occurs at MIN node C!
- The remaining children of C (nodes **H** and **I**) are **pruned**.
- Thus, we do not examine leaves **R** (9), **S** (5), **T** (3), **U** (1).
- $Value(C) = 2$.

Branch D (Right Side of Root): We search with $\alpha = 4$ and $\beta = +\infty$.

1. Analyze Node J:

- Children: V (5) and W (4).
- Since J is a MAX node, $Value(J) = \max(5, 4) = 5$.
- Propagate to D (MIN node): β_D becomes $\min(+\infty, 5) = 5$.
- At this point, $\alpha_D = 4$ and $\beta_D = 5$. (No pruning yet).

1. Analyze Node K:

- Children: X (7).
- **Check Pruning Condition at K:**
- K is a MAX node, currently evaluating child X (7).
- $Value(K) \geq 7$.

- Since D is a MIN node above K, D's beta $\beta_D = 5$.
- If K's value is ≥ 7 , D (MIN) will definitely choose J (5) over K (≥ 7).
- Therefore, since $Value(K) \geq 7 > \beta_D$ ($7 \geq 5$), a **Beta-Cutoff** occurs at MAX node K!
- The remaining child of K (node Y (5)) is **pruned**.

1. **Evaluate Node D:**

- $Value(D) = \min(Value(J), Value(K)) = \min(5, 7) = 5$.

Evaluate Root A:

- $Value(A) = \max(Value(B), Value(C), Value(D)) = \max(4, 2, 5) = 5$.
- **Optimal Move for Maximizer:** Choose move to **Node D**.

Final Audit Summary:

- **Minimax values of intermediate nodes:**
 - $E = 4, F = 6, G = 2, J = 5, K = 7$.
 - $B = 4, C = 2, D = 5, A = 5$.
- **Pruned Branches:**
 - Nodes **H** and **I** (and their children R, S, T, U) via **Alpha-Cutoff** at node C.
 - Leaf **Y** via **Beta-Cutoff** at node K.
- **Number of terminal leaves examined:** 9 out of 14 leaves (L, M, N, O, P, Q, V, W, X).

Chapter 7

Logical Agents

7.1 Section 1: Propositional Logic Definitions and Tautology Proof (Q7.1)

7.1.1 1. Key Definitions in Propositional Logic

- **Tautology:** A logical sentence that is true under every possible truth assignment of its variables. It represents a statement that is logically, necessarily true (e.g., $P \vee \neg P$).
- **Contradiction (Unsatisfiable):** A logical sentence that is false under every possible truth assignment of its variables. It represents a statement that is logically impossible (e.g., $P \wedge \neg P$).
- **Contingency:** A logical sentence that can be either true or false depending on the truth values assigned to its variables. Most everyday propositions are contingencies (e.g., $P \wedge Q$).

7.1.2 2. Proof: $((P \rightarrow Q) \rightarrow P) \rightarrow P$ is a Tautology

This formula is known as **Peirce's Law**. We construct a truth table to evaluate its truth value across all possible combinations of P and Q .

Row	P	Q	$P \rightarrow Q$	$(P \rightarrow Q) \rightarrow P$	$((P \rightarrow Q) \rightarrow P) \rightarrow P$
1	T	T	T	T	T
2	T	F	F	T	T
3	F	T	T	F	T
4	F	F	T	F	T

Explanation of Steps:

Column 4 ($P \rightarrow Q$):* False only when P is True and Q is False (Row 2). Column 5 $((P \rightarrow Q) \rightarrow P)$:* Implication is false only if the premise $((P \rightarrow Q))$ is True and the conclusion (P) is False. This happens in Row 3 ($T \rightarrow F$) and Row 4 ($T \rightarrow F$). Column 6 $((P \rightarrow Q) \rightarrow P) \rightarrow P$:* Implication is false only if Column 5 is True and P is False. Let's verify:

- Row 1: $T \rightarrow T \Rightarrow$ True.
- Row 2: $T \rightarrow T \Rightarrow$ True.
- Row 3: $F \rightarrow F \Rightarrow$ True.
- Row 4: $F \rightarrow F \Rightarrow$ True.

Conclusion: Since the final column is True in all possible rows, the sentence $((P \rightarrow Q) \rightarrow P) \rightarrow P$ is proved to be a **Tautology**. ■

7.2 Section 2: Logical Inference Rules (Q7.2)

Inference rules are patterns of logical deduction that derive a conclusion from premises.

7.2.1 1. Modus Ponens

- **Syntax:**

$$\frac{P, \quad P \rightarrow Q}{Q}$$

- **Description:** If both P and "If P then Q " are true, then Q must be true.
- **Truth Table Validation:**
- We show that the sentence $((P \wedge (P \rightarrow Q)) \rightarrow Q)$ is a Tautology.

P	Q	$P \rightarrow Q$	$P \wedge (P \rightarrow Q)$	$(P \wedge (P \rightarrow Q)) \rightarrow Q$
T	T	T	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	T

7.2.2 2. Modus Tollens

- **Syntax:**

$$\frac{\neg Q, \quad P \rightarrow Q}{\neg P}$$

- **Description:** If P implies Q , and Q is false, then P must be false.
- **Truth Table Validation:**
- We show that the sentence $((\neg Q \wedge (P \rightarrow Q)) \rightarrow \neg P)$ is a Tautology.

P	Q	$\neg P$	$\neg Q$	$P \rightarrow Q$	$\neg Q \wedge (P \rightarrow Q)$	$(\neg Q \wedge (P \rightarrow Q)) \rightarrow \neg P$
T	T	F	F	T	F	T
T	F	F	T	F	F	T
F	T	T	F	T	F	T
F	F	T	T	T	T	T

7.2.3 3. Resolution Rule

- **Syntax:**

$$\frac{P \vee Q, \quad \neg P \vee R}{Q \vee R}$$

- **Description:** If we know that P or Q is true, and we also know that either $\neg P$ or R is true, then we can resolve the conflict to conclude that Q or R must be true.
- **Unit Resolution (Special Case where R is empty):**

$$\frac{P \vee Q, \quad \neg P}{Q}$$

7.3 Section 3: Knowledge-Based Agents and Wumpus World (Q7.3)

7.3.1 1. Knowledge-Based Agents

A **Knowledge-Based Agent** is an agent whose internal state is defined by a **Knowledge Base (KB)** – a set of representations of facts (sentences) about the world expressed in a formal logic language.

The agent interacts with its KB using two main functions:

- **‘TELL’**: Adds a new sentence (percept information or inferred fact) to the knowledge base.
- **‘ASK’**: Queries the knowledge base to determine what action should be taken next.

The KB-Agent Working Loop:

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
               t, a counter, initially 0

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action <- ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t <- t + 1
  return action
```

7.3.2 2. The Wumpus World Environment

The **Wumpus World** is a grid environment used to demonstrate logical reasoning. An agent must navigate the grid to find a heap of gold and return safely without falling into a pit or being eaten by the beast (the Wumpus).

Wumpus World 4x4 Grid				
	4 Stench	Breeze	PIT	
	3 Agent	Breeze	PIT	Breeze
	2 Stench	Breeze		
	1 Start	Breeze	PIT	Breeze
	1	2	3	4

Figure 7.1: Wumpus World Grid Layout

PEAS Description of Wumpus World:

- **P (Performance)**: +1000 for climbing out of the cave with gold; −1000 for falling into a pit or being eaten; −1 for each action taken; −10 for using the arrow.

- **E (Environment):** 4×4 grid of rooms. Pits are placed randomly (each cell has 0.2 probability of containing a pit). The Wumpus is in a single random cell. Gold is in a single random cell.
- **A (Actuators):** Turn Left, Turn Right, Forward, Grab (gold), Shoot (arrow), Climb (out).
- **S (Sensors):**

Stench:* Felt in cells adjacent to the Wumpus. Breeze:* Felt in cells adjacent to pits. Glitter:* Felt in the cell containing the gold. Bump:* Felt when walking into a wall. Scream:* Heard anywhere in the cave when the Wumpus is killed.

7.3.3 3. Step-by-Step Safe Logic Reasoning Example

Let the agent start at cell (1,1). Let's trace how the agent reasons using logic rules to move safely:

1. **Initial Percept at (1,1):** '[None, None, None, None, None]' (No Stench, No Breeze).

Logical Rule:* If there is no Breeze at (1,1), then there is no pit in (1,2) and (2,1). Logical Rule:* If there is no Stench at (1,1), then the Wumpus is not in (1,2) or (2,1). Deduction:* Cells (1,2) and (2,1) are safe.

1. **Move to (2,1):**

Percept at (2,1):* '[None, Breeze, None, None, None]' (Breeze detected). Logical Rule:* A Breeze at (2,1) implies a pit exists in one of the adjacent cells: (1,1), (2,2), or (3,1). Deduction:* Since (1,1) is the start and is safe, the pit is either in (2,2) or (3,1). The agent cannot determine which one is the pit yet, so it backtracks.

1. **Return to (1,1) and move to (1,2):**

Percept at (1,2):* '[Stench, None, None, None, None]' (Stench detected, No Breeze). Logical Rule:* Since there is no Breeze at (1,2), adjacent cells (1,3) and (2,2) have no pits. Deduction:*

- Since (2,2) has no pit, the pit detected from (2,1) must be in (3,1).
- A Stench at (1,2) implies the Wumpus is in (1,1), (1,3), or (2,2).
- Since (1,1) is safe, and (2,2) has no Stench (verified earlier from (1,1)), the Wumpus must be in (1,3).

Resulting Safe Plan:* Cell (2,2) is free of both pits and Wumpus, and is safe to enter next.

7.4 Section 4: Propositional Semantics, Logical Inference, and Resolution (Q7.4) [5M][]

Logical agents rely on propositional logic semantics to evaluate truth, compute inferences, and prove theorems.

7.4.1 1. Semantics and Proposition Symbols

- **Semantics:** The **semantics of propositional logic** defines the rules for computing the truth value of any complex sentence in a given model (an assignment of truth values to proposition variables).
- **Proposition Symbols:**
 - In AI propositional logic, there are exactly **2 standard proposition symbols** that represent constant truth values: **True** ($\top$, representing absolute truth) and **False** ($\perp$, representing absolute falsity).
 - Unlike user-defined propositional variables (e.g., P, Q, R), which can be assigned arbitrary truth values, these two constant symbols have fixed, immutable semantics.

7.4.2 2. Logical Inference Properties

To compute logical inferences ($KB \models \alpha$), inference engines utilize three key mathematical properties:

1. **Validity:** A sentence is valid if it is true in **all possible models** (a Tautology). Validity is used because $KB \models \alpha$ if and only if the implication ($KB \rightarrow \alpha$) is valid.
2. **Satisfiability:** A sentence is satisfiable if it is true in **at least one model**. Satisfiability is connected to inference via refutation: $KB \models \alpha$ if and only if the sentence ($KB \wedge \neg\alpha$) is **unsatisfiable** (a contradiction).
3. **Logical Equivalence:** Two sentences α and β are logically equivalent ($\alpha \equiv \beta$) if they share the same truth value in every model. This property allows syntactic transformation of sentences without altering their semantic truth.

7.4.3 3. CNF Equivalence & Clause Terminology

- **CNF Equivalence:** Every sentence of propositional logic can be converted into an inferred equivalent **Conjunctive Normal Form (CNF)** sentence (a conjunction of disjunctions of literals). This is because standard logical operators ($\rightarrow, \leftrightarrow$) can always be rewritten, negations pushed inward, and disjunctions distributed over conjunctions using equivalences that preserve truth values.
- **Original Statement Satisfiability:** If a converted CNF sentence is unsatisfiable, the **original statement** is unsatisfiable.
- **Unit Clause:** A **Unit Clause** is a clause containing exactly **a single literal** (e.g., $\{P\}$ or $\{\neg Q\}$). It represents a single literal of disjunction (a disjunction of one element). Unit clauses are highly valuable in resolution algorithms because they allow **Unit Resolution**, which simplifies search complexity significantly.

7.4.4 4. Resolution as a Single Inference Rule

- **The Resolution Rule:**

$$\frac{l_1 \vee \dots \vee l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee \neg l_i \vee \dots \vee m_n}{l_1 \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_n} \quad (\text{with } l_i \text{ and } \neg l_i \text{ resolved})$$

- **Single Inference Rule:** Resolution is called a **single inference rule** because it is **refutation-complete**. Combined with a search algorithm, it is the only rule needed to prove any logical inference. We do not need a large set of individual rules (such as Modus Ponens, Modus Tollens, addition, or elimination rules); the resolution rule alone can deduce contradiction ($\square$) from any unsatisfiable set of clauses, making inference engines simpler and more efficient to implement.

Chapter 8

First-Order Logic & Knowledge Representation

8.1 Section 1: Declarative vs. Procedural Knowledge and Production Systems (Q8.1)

8.1.1 1. Comparison of Knowledge Types

Feature	Declarative Knowledge	Procedural Knowledge
Definition	Knowledge that represents static facts, assertions, and concepts about the world.	Knowledge that represents the steps, rules, and procedures to perform a task.
Nature	"Knowing what" (Passive, descriptive facts).	"Knowing how" (Active, procedural steps).
Representation	First-Order Logic, semantic networks, database schemas, frames.	Rules, algorithms, subroutines, step-by-step procedures.
Modifiability	Very easy to add, delete, or update facts independently.	Hard to modify; changes in one step can break subsequent steps.
Execution	Needs an external interpreter or inference engine to process.	Executable directly by the processor/system.

8.1.2 2. Production Systems in Knowledge Representation

A **Production System** is a modular cognitive model of problem-solving consisting of three main components:

1. **Production Rules (Rule Base):** A set of IF-THEN rules of the form:

$$\text{IF } \langle \text{Condition/Pattern} \rangle \longrightarrow \text{THEN } \langle \text{Action} \rangle$$

1. **Working Memory (Database):** Holds the current facts and assertions representing the system's current state.
2. **Recognize-Act Cycle (Inference Engine):**

Match:* Compares working memory facts against the IF conditions of production rules. Conflict Resolution:* If multiple rules match, a conflict resolution strategy (e.g., recency, specificity, rule priority) selects one rule. Act:* Executes the action part of the selected rule, updating the working memory.

8.2 Section 2: Horn Clauses and Skolemisation (Q8.2)

8.2.1 1. Horn Clause

A **Horn Clause** is a disjunction of literals (clause) containing **at most one positive (un-negated) literal**. It is the foundation of logic programming (like PROLOG).

There are three types of Horn clauses:

1. **Definite Clause:** Exactly one positive literal (e.g., $\neg p \vee \neg q \vee r$, which is equivalent to $(p \wedge q) \rightarrow r$).
2. **Fact:** A single positive literal with no negative literals (e.g., p).
3. **Goal Clause:** No positive literals (e.g., $\neg p \vee \neg q$, equivalent to $\neg(p \wedge q)$).

Proof: $p \rightarrow q$ is a **Horn Clause**

By logical equivalence, the implication $p \rightarrow q$ can be written as:

$$\neg p \vee q$$

This is a disjunction of literals containing exactly one positive literal (q) and one negative literal ($\neg p$). By definition, since it contains at most one positive literal, $p \rightarrow q$ is a **Horn Clause** (specifically, a definite clause). ■

8.2.2 2. Skolemisation

Skolemisation is the process of eliminating existential quantifiers ($\exists$) in a First-Order Logic formula, replacing them with concrete terms (constants or functions) without changing the satisfiability of the formula.

Rules for Skolemisation:

1. **If the existential quantifier is NOT within the scope of any universal quantifier ($\forall$):** Replace the variable with a new, unique constant (a **Skolem Constant**).

Example:* $\exists x P(x) \implies P(A)$ (where A is a new constant).

1. **If the existential quantifier lies within the scope of one or more universal quantifiers ($\forall$):** Replace the existential variable with a new function (a **Skolem Function**) whose arguments are the universally quantified variables.

Example:* $\forall x \exists y \text{Loves}(x, y) \implies \forall x \text{Loves}(x, F(x))$ (where $F(x)$ is a new function mapping each x to its corresponding y).

8.3 Section 3: PROLOG Programming (Q8.3)

8.3.1 1. Recursive Factorial Program

```
% Base Case: Factorial of 0 is 1.
factorial(0, 1).

% Recursive Case: Factorial of N (N > 0) is F.
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

8.3.2 2. Recursive Greatest Common Divisor (GCD) Program

```
% Base Case: GCD of X and 0 is X.
gcd(X, 0, X) :- !.

% Recursive Case: GCD(X, Y) using Euclidean algorithm (modulo operator)
gcd(X, Y, G) :-
    Y > 0,
    R is X mod Y,
    gcd(Y, R, G).
```

8.4 Section 4: Clausal Form (CNF) and Resolution Refutation Proofs (Q8.4)

8.4.1 1. Step-by-Step Procedure to Convert to CNF

1. **Eliminate Implications & Biconditionals:** Replace $A \Rightarrow B$ with $\neg A \vee B$.
2. **Move Negations Inward:** Apply De Morgan's laws and double-negation rules (e.g., $\neg(A \wedge B) \equiv \neg A \vee \neg B$, $\neg\forall x P(x) \equiv \exists x \neg P(x)$).
3. **Standardize Variables:** Rename variables so that each quantifier binds a unique variable name.
4. **Skolemise:** Eliminate existential quantifiers ($\exists$) using Skolem constants/functions.
5. **Drop Universal Quantifiers ($\forall$):** Since all remaining variables are universally quantified, we can drop the quantifiers.
6. **Distribute $\vee$ over $\wedge$:** Structure into Conjunctive Normal Form (CNF) (e.g., $(A \vee B) \wedge (C \vee D)$).
7. **Isolate Clauses:** Write each conjunct as a separate clause.

8.4.2 2. CNF Conversion Example

Convert the following to CNF:

$$(\forall x)(P(x) \Rightarrow ((\forall y)(P(y) \Rightarrow P(f(x, y))) \wedge \neg(\forall y)(Q(x, y) \Rightarrow P(y))))$$

- **Step 1: Eliminate implications**

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x, y))) \wedge \neg(\forall y)(\neg Q(x, y) \vee P(y))))$$

- **Step 2: Move negations inward**

- Focus on $\neg(\forall y)(\neg Q(x, y) \vee P(y))$:

$$\neg(\forall y)(\dots) \implies \exists y \neg(\neg Q(x, y) \vee P(y)) \implies \exists y(Q(x, y) \wedge \neg P(y))$$

- Reassemble:

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x, y))) \wedge \exists y(Q(x, y) \wedge \neg P(y))))$$

- **Step 3: Standardize variables**

Rename the second y to z to prevent overlap:

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x, y))) \wedge \exists z(Q(x, z) \wedge \neg P(z))))$$

- **Step 4: Skolemise**

The existential variable z is within the scope of universal quantifier x . Replace z with Skolem function $h(x)$:

$$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x, y))) \wedge (Q(x, h(x)) \wedge \neg P(h(x)))))$$

- **Step 5: Drop universal quantifiers**

$$\neg P(x) \vee ((\neg P(y) \vee P(f(x, y))) \wedge Q(x, h(x)) \wedge \neg P(h(x)))$$

- **Step 6: Distribute $\vee$ over $\wedge$**

Apply distributive law: $A \vee (B \wedge C \wedge D) \equiv (A \vee B) \wedge (A \vee C) \wedge (A \vee D)$.

- **Clause 1:** $\neg P(x) \vee \neg P(y) \vee P(f(x, y))$
- **Clause 2:** $\neg P(x) \vee Q(x, h(x))$
- **Clause 3:** $\neg P(x) \vee \neg P(h(x))$

8.4.3 3. Resolution Refutation Proof Case Study

Statements:* "Everyone who enters in a theatre has bought a ticket. Person who does not have money can't buy a ticket. Vinod enters a theatre." Goal:* Prove that "Vinod has money" ($HasMoney(Vinod)$).

FOPL Representation:

1. "Everyone who enters in a theatre has bought a ticket":

$$\forall x(Enters(x) \rightarrow BoughtTicket(x)) \implies \neg Enters(x) \vee BoughtTicket(x)$$

1. "Person who does not have money can't buy a ticket":

$$\forall y(\neg HasMoney(y) \rightarrow \neg BoughtTicket(y)) \implies \text{Implication: } BoughtTicket(y) \rightarrow HasMoney(y) \implies \neg BoughtTicket(y) \rightarrow \neg HasMoney(y)$$

1. "Vinod enters a theatre":

$$Enters(Vinod)$$

1. **Negated Goal:**

$$\neg HasMoney(Vinod)$$

Resolution Proof:

Step 1:* Resolve Clause 1 ($\neg Enters(x) \vee BoughtTicket(x)$) and Clause 3 ($Enters(Vinod)$) using substitution $\{x/Vinod\}$:

- **Inferred Clause 5:** $BoughtTicket(Vinod)$

Step 2:* Resolve Clause 2 ($\neg BoughtTicket(y) \vee HasMoney(y)$) and Clause 5 ($BoughtTicket(Vinod)$) using substitution $\{y/Vinod\}$:

- **Inferred Clause 6:** $HasMoney(Vinod)$

Step 3:* Resolve Clause 6 ($HasMoney(Vinod)$) and Negated Goal ($\neg HasMoney(Vinod)$):

- **Inferred Clause 7:** $\square$ (Empty clause / Contradiction).
- **Conclusion:** Since the empty clause is derived, the negated goal is false. Thus, **Vinod has money.** ■

8.5 Section 5: Predicate Logic Case Study – Marcus Hated Caesar (Q8.5)

8.5.1 1. FOPL Representation

1. *Marcus was a man:* $Man(Marcus)$
2. *Marcus was Pompeian:* $Pompeian(Marcus)$
3. *All Pompeians were Romans:* $\forall x(Pompeian(x) \rightarrow Roman(x)) \Rightarrow \neg Pompeian(x) \vee Roman(x)$
4. *Caesar was a ruler:* $Ruler(Caesar)$
5. *All Romans were either loyal to Caesar or hated him:*

$$\forall x(Roman(x) \rightarrow LoyalTo(x, Caesar) \vee Hate(x, Caesar)) \Rightarrow \neg Roman(x) \vee LoyalTo(x, Caesar) \vee Hate(x, Caesar)$$

1. *Everyone is loyal to someone:* $\forall x \exists y LoyalTo(x, y) \Rightarrow \forall x LoyalTo(x, f(x))$ (where $f(x)$ is a Skolem function).
2. *People only try to assassinate rulers they are not loyal to:*

$$\begin{aligned} &\forall x \forall y (Man(x) \wedge Ruler(y) \wedge TryAssassinate(x, y) \rightarrow \neg LoyalTo(x, y)) \\ &\Rightarrow \neg Man(x) \vee \neg Ruler(y) \vee \neg TryAssassinate(x, y) \vee \neg LoyalTo(x, y) \end{aligned}$$

1. *Marcus tried to assassinate Caesar:* $TryAssassinate(Marcus, Caesar)$

8.5.2 2. Resolution Refutation Proof

- **Negated Goal:** $\neg Hate(Marcus, Caesar)$

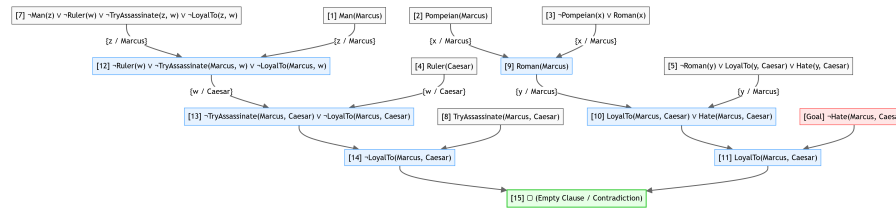


Figure 8.1: Resolution Refutation Proof Tree

Conclusion: The contradiction proves that the negated goal is false. Therefore, **Marcus hated Caesar.** ■

8.6 Section 6: Knowledge Representation Techniques (Q8.6)

8.6.1 1. Semantic Networks

- **Definition:** A graphical representation of knowledge consisting of nodes (objects/concepts) and directed, labeled edges (relations).

Sourav is 6 feet tall and is taller than Sachin.*

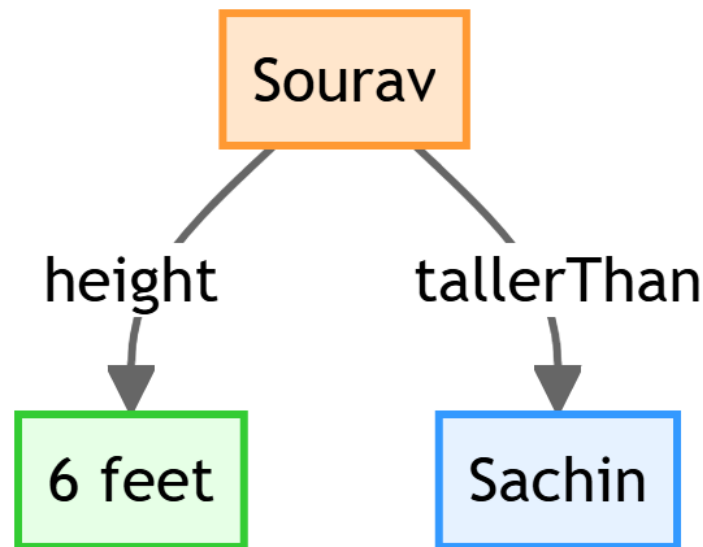


Figure 8.2: Semantic Network

8.6.2 2. Frames

- **Definition:** A collection of attributes (called **slots**) and associated values (called **fillers**) that describe an entity. Support class-inheritance.
- **Example Frame:**

```

Frame: Patient
  isa: Person
  Age: default_value (30)
  Doctor: default_value (Dr. Smith)

Frame: John
  isa: Patient
  Age: 45
  (Inherits Doctor as Dr. Smith; overrides
    Age to 45)
  
```

8.6.3 3. Fuzzy Logic

- **Definition:** A form of many-valued logic where truth values are real numbers between 0 (completely false) and 1 (completely true), representing degrees of membership in a set.
- **Fuzzy Set Operations:**
- **Union ($\cup$):** $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- **Intersection ($\cap$):** $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- **Complement ($\bar{A}$):** $\mu_{\bar{A}}(x) = 1 - \mu_A(x)$
- **Crisp vs. Fuzzy Set Difference:** A crisp set has binary membership ($\mu(x) \in \{0, 1\}$; an element is either in the set or not), while a fuzzy set allows continuous degrees of membership ($\mu(x) \in [0, 1]$; e.g., a person can be "partially tall" with membership 0.7).

8.6.4 4. Neural Networks and Genetic Algorithms

- **Hebb's Rule:** A learning rule stating that the weight of a connection between two neurons increases if both neurons are active at the same time ("Neurons that fire together, wire together"):

$$\Delta w_{ij} = \eta \times a_i \times a_j$$

Where η is learning rate, and a_i, a_j are neuron activations.

- **Genetic Algorithms Operators:**
- **Crossover:** Combines genetic material from two parent chromosomes to create offspring (e.g., cutting parents at a point and swapping tails).
- **Mutation:** Randomly alters one or more gene values in a chromosome (e.g., flipping a bit from 0 to 1) with low probability to maintain genetic diversity.

Chapter 9

Quick Revision: OE-EC804A - Artificial Intelligence

This high-yield index contains key definitions, formulas, repeated questions, practice diagrams, and last-minute revision points to review before exams.

9.1 Unit I: Introduction

9.1.1 1. Important Definitions

- **Artificial Intelligence:** Autonomous systems performing cognitive tasks that typically require human intelligence.
- **Rational Agent:** An agent that acts to achieve the best possible (or best expected) outcome based on its inputs and knowledge.
- **Turing Test:** An operational test where a computer is considered intelligent if a human interrogator cannot distinguish it from a human in textual conversations.
- **Expert System:** Specialized interactive software emulating a human expert's decision-making in a narrow domain.
- **Inference Engine:** The reasoning logic in an Expert System that applies facts/rules to draw conclusions.
- **Forward Chaining:** Data-driven reasoning starting from known facts to reach a conclusion.
- **Backward Chaining:** Goal-driven reasoning starting from a hypothesis/goal to find supporting facts.
- **Knowledge Acquisition Bottleneck:** The slow, difficult task of extracting human expertise into rigid computer rules.

9.1.2 Key Formulas

None for Unit I. (Note: Search complexities and Bayes probability formulas appear in later units).*

9.1.3 Frequently Asked Questions

1. "Describe different components of AI." [5M]

Core point:* Focus on NLP, Machine Learning, Computer Vision, Robotics, and Expert Systems. Include the Sensing → Reasoning → Learning → Acting functional diagram.

1. "What is an Expert System? Explain its components." [10M]

Core point:* Define ES. Draw the Knowledge Base ↔ Inference Engine ↔ User Interface block diagram. Detail forward/backward chaining. List 5 advantages and 5 disadvantages.

9.1.4 Diagrams to Practice

1. The Turing Test Setup:

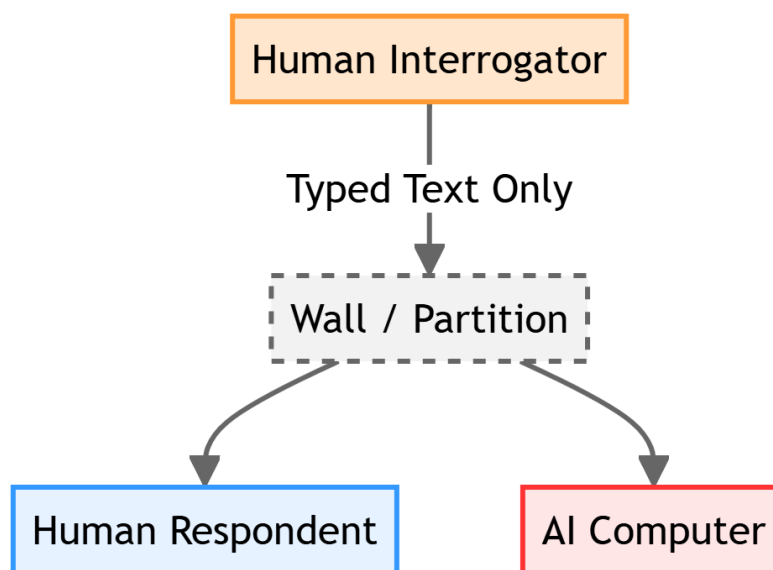


Figure 9.1: Turing Test Setup

1. Expert System Block Diagram:

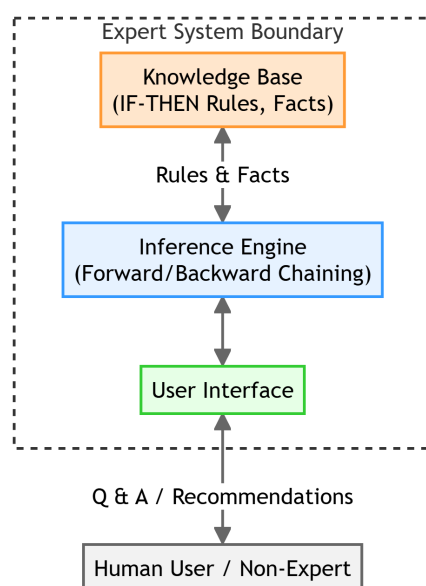


Figure 9.2: Expert System Architecture

9.1.5 One-Line Revision

- AI is divided into 4 views: Thinking/Acting Humanly (behavioral) and Thinking/Acting Rationally (performance-driven).
- Acting Rationally (Rational Agent approach) is the standard modern paradigm of AI studies.
- The Turing Test requires NLP, Knowledge Representation, Automated Reasoning, and Machine Learning.
- The Total Turing Test adds Computer Vision and Robotics.
- Expert Systems use rigid **IF-THEN production rules** for representation.
- Forward Chaining is **data-driven**; Backward Chaining is **goal-driven**.

9.2 Unit II: Intelligent Agents

9.2.1 1. Important Definitions

- **Agent:** Anything that perceives its environment through sensors and acts upon it through actuators.
- **Percept Sequence:** The complete history of all percepts received by the agent to date.
- **Agent Function (f):** A mathematical mapping $f : P^* \rightarrow A$ from any percept sequence to an action.
- **Agent Program:** The concrete implementation of the agent function running on the physical architecture.
- **PEAS:** Performance measure, Environment, Actuators, Sensors. Used to specify the task environment.
- **Rational Agent:** An agent that selects actions to maximize its expected performance measure given its percept history and knowledge.

- **Autonomy:** An agent behaves with autonomy if its behavior is determined by its own experience rather than depending solely on built-in designer knowledge.

9.2.2 Key Formulas

- Agent = Architecture + Program
- $f : P^* \rightarrow A$ (Agent Function mapping percept history to action)

9.2.3 Frequently Asked Questions

1. "Define PEAS description for an automated taxi driver." [5M][[]]

Core point:* P: Safety, speed, comfort, profits. E: Roads, traffic, pedestrians. A: Steering, gas, brakes, display. S: Cameras, LIDAR, GPS, speedometer.

1. "Explain the five basic agent structures." [15M][[]]

Core point:* Simple Reflex (percept-action rules), Model-Based (uses state to track unobserved world), Goal-Based (uses goals to plan), Utility-Based (uses utility function for state preference), Learning Agent (Critic, Learning Element, Performance Element, Problem Generator).

9.2.4 Diagrams to Practice

1. Simple Reflex and Model-Based Agents:

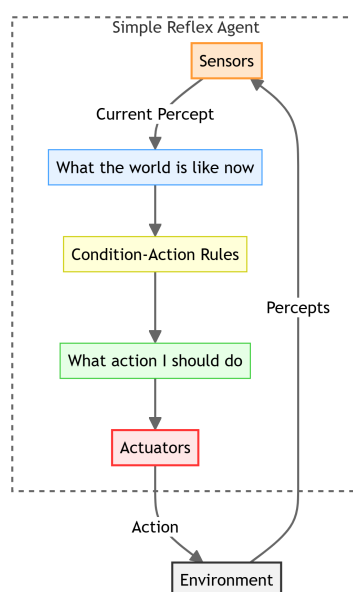


Figure 9.3: Simple Reflex Agent Structure

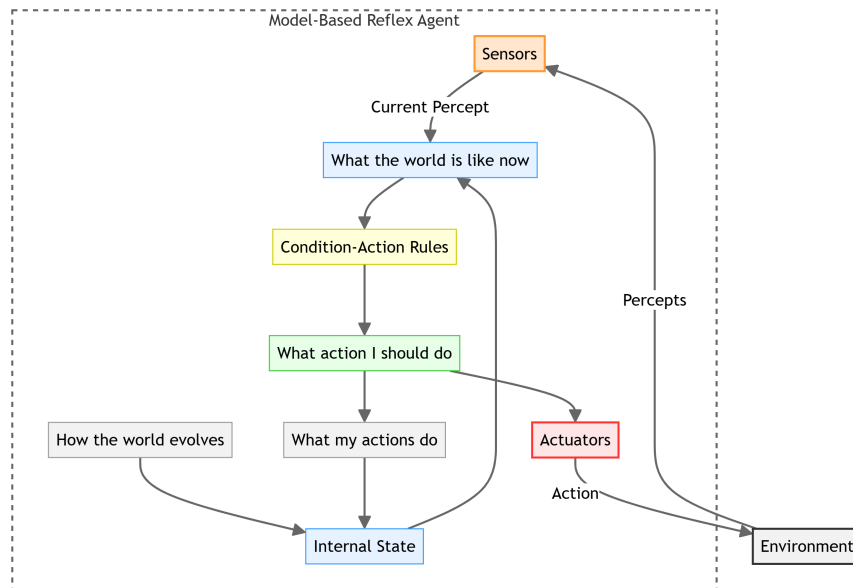


Figure 9.4: Model-Based Reflex Agent Structure

1. Learning Agent Architecture:

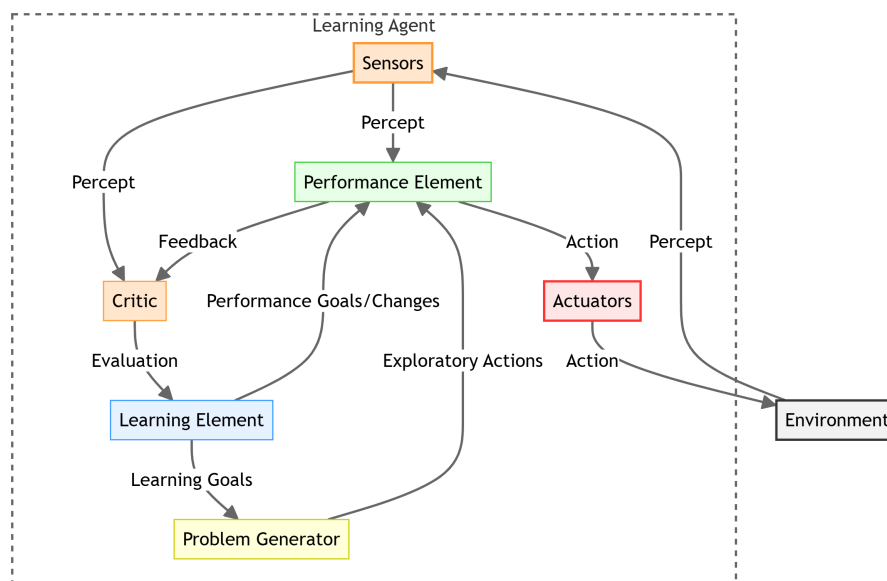


Figure 9.5: Learning Agent Structure

9.2.5 One-Line Revision

- An agent acts using actuators and senses using sensors.
- A simple reflex agent uses current percepts only; model-based reflex agents use internal states to track the unobserved environment.
- Goal-based agents plan to achieve goals; utility-based agents maximize a real-valued performance metric.
- A learning agent separates the critic (evaluator) from the learning element (improver) and the problem generator (explorer).

- Environments can be fully/partially observable, deterministic/stochastic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent.

9.3 Unit III: Solving Problems by Searching (Uninformed Search)

9.3.1 1. Important Definitions

- **Uninformed Search (Blind Search):** Search algorithms that do not use any problem-specific knowledge (heuristics) to guide the search process. They only use the problem definition to explore the state space.
- **State Space:** The set of all possible configurations (states) the problem can be in.
- **Initial State:** The starting configuration of the problem.
- **Goal State:** The desired configuration that the search aims to find.
- **Operators (Successor Function):** Actions that can be applied to a state to transition to a new state.
- **Path:** A sequence of states from the initial state to a goal state.
- **Cost:** The cost associated with applying an operator (usually 1 for unweighted graphs).

9.3.2 2. Characteristics of Uninformed Search

- **No Heuristics:** Does not use any problem-specific knowledge to guide the search.
- **Systematic Exploration:** Explores the state space systematically to find a solution.
- **Completeness:** Guaranteed to find a solution if one exists (depending on the algorithm).
- **Optimality:** Guaranteed to find the optimal (lowest cost) solution if one exists (depending on the algorithm).
- **Memory Usage:** Can be memory-intensive as it may need to store large portions of the state space.

9.3.3 3. Important Search Algorithms

1. Breadth-First Search (BFS):

- **Principle:** Explores the state space level by level. It expands all nodes at the current depth before moving to the next depth level. Uses a queue data structure.
- **Completeness:** Yes, if a solution exists and the branching factor is finite.
- **Optimality:** Yes, if the cost of each step is uniform (e.g., 1). Finds the shortest path in terms of number of edges.
- **Time/Space Complexity:** $O(b^d)$, where b is branching factor and d is depth.

1. Depth-First Search (DFS):

- **Principle:** Explores as far as possible along each branch before backtracking. Uses a stack data structure.
- **Completeness:** No, fails in infinite paths or cycles.

- **Optimality:** No.
- **Time/Space Complexity:** Time: $O(b^m)$, Space: $O(bm)$ (where m is max depth).

1. Depth-Limited Search (DLS):

- **Principle:** DFS with a depth limit l imposed.

1. Iterative Deepening Search (IDDFS):

- **Principle:** Performs DLS with increasing limits $l = 0, 1, 2, \dots$. Combines DFS space efficiency $O(bd)$ with BFS completeness and optimality.

1. Uniform Cost Search (UCS):

- **Principle:** Expands the node with the lowest path cost $g(n)$ using a priority queue. Optimal for arbitrary step costs.

9.3.4 4. Algorithm Properties

Property	BFS	DFS	DLS	IDDFS	UCS
Completeness	Yes	No	No	Yes	Yes
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$
Space Complexity	$O(b^d)$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$
Optimality	Yes (uniform)	No	No	Yes (uniform)	Yes

9.3.5 Diagrams to Practice

1. Water Jug Solution Path:

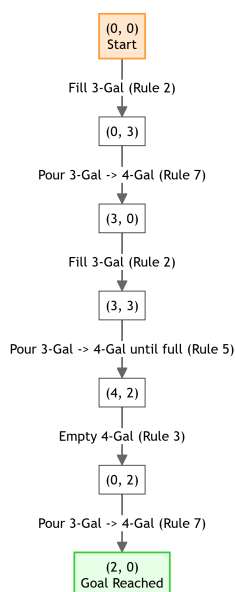


Figure 9.6: Water Jug Traced Solution Path

9.4 Unit IV: Informed Search & Exploration

9.4.1 1. Important Definitions

- **Informed Search (Heuristic Search):** Search algorithms that use problem-specific knowledge (heuristics) to guide the search process. They use an evaluation function to estimate the cost from the current state to the goal state.
- **Heuristic Function ($h(n)$):** An informed guess about the cost from the current state to the goal state. It is used to guide the search process.
- **Evaluation Function ($f(n)$):** A function that evaluates the cost of a state. It is used to guide the search process.
- **Recursive Best-First Search (RBFS):** An optimal, best-first heuristic search algorithm that runs in linear space ($O(bd)$) by using recursive depth-first execution and maintaining an f -limit value for backtracking.

9.4.2 2. Important Algorithms

1. Greedy Best-First Search:

- **Principle:** Expands the node that appears to be closest to the goal state, as estimated by the heuristic function.
- **Evaluation Function:** $f(n) = h(n)$
- **Completeness/Optimality:** No.

1. A* Search:

- **Principle:** Expands the node that appears to be closest to the goal state, as estimated by the evaluation function $f(n)$.
- **Evaluation Function:** $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start state to the current state, and $h(n)$ is the estimated cost from the current state to the goal state.
- **Completeness/Optimality:** Yes, if $h(n)$ is admissible (tree search) or consistent (graph search).

1. Local Search and Optimization Techniques:

- **Hill-Climbing Search:** Greedy algorithm that moves to the neighboring state with the highest heuristic value. Fails at local maxima, ridges, and plateaus.
- **Local Beam Search:** Tracks k states instead of just one, expanding all successors and keeping the best k .

AO Search: Used for search trees representing AND-OR graphs (decomposable problems).

9.4.3 Diagrams to Practice

1. Heuristic Consistency Triangle:

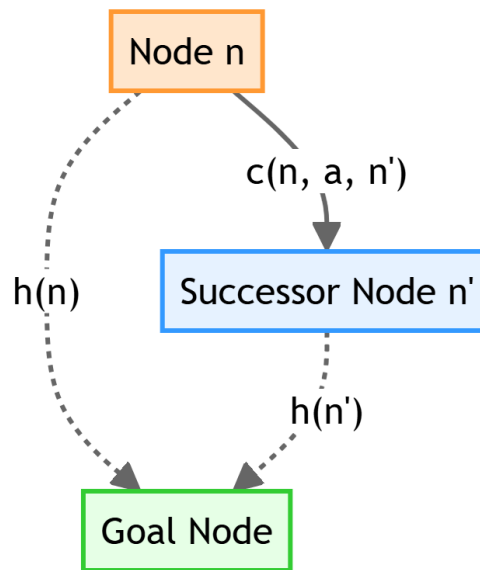


Figure 9.7: Heuristic Consistency Inequality Triangle

1. AND-OR Graph Decompositions:

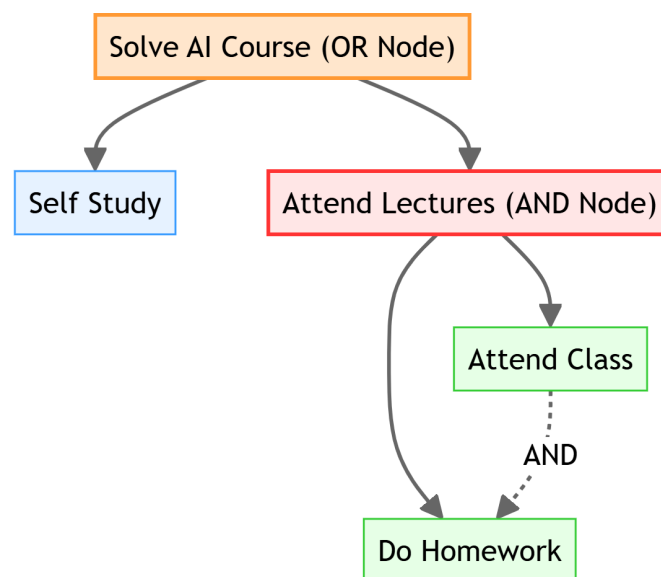


Figure 9.8: AND-OR Graph Decompositions

9.4.4 One-Line Revision

- Greedy Best-First Search evaluates nodes using $f(n) = h(n)$, expanding the node estimated to be closest to the goal.
- The space complexity of Greedy Search is exponential ($O(b^l)$) because it stores all frontier nodes.
- RBFS reduces the space complexity to linear ($O(bd)$) but can have exponential time complexity due to node regeneration.

9.5 Unit V: Constraint Satisfaction Problems

9.5.1 1. Important Definitions

- **Constraint Satisfaction Problem (CSP):** A problem defined by a set of variables, domains (allowed values), and constraints restricting value combinations.
- **Backtracking Search:** DFS-based search for CSPs where variables are assigned sequentially and reverted on constraint violation.
- **Alldifferent Constraint:** A global constraint specifying that all variables in a set must be assigned unique values.
- **Flexible CSP:** A CSP formulation where constraints are relaxed (preferences/soft constraints) to allow a solution when the problem is overconstrained.

9.5.2 Frequently Asked Questions

1. "Solve the cryptarithmic problem: $\text{SEND} + \text{MORE} = \text{MONEY}$." [15M]

Core point:* Set up variables, domains, and column carry constraints. Solve sequentially: $M = 1$ (carry), $O = 0$, $S = 9$, $C_3 = 0$, $C_2 = 1$, $N = E + 1$. Find remaining digits: $R = 7$, $E = 5$, $N = 6$, $D = 8$, $Y = 3$.

9.5.3 Diagrams to Practice

1. Australia Constraint Graph:

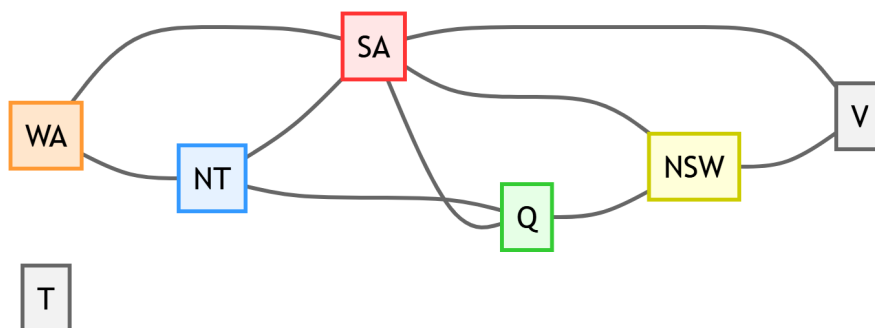


Figure 9.9: Australia Map-Coloring Outline

9.5.4 One-Line Revision

- CSPs use a factored state representation (variables and constraints) instead of atomic black boxes.
- Backtracking is the standard complete search algorithm for CSPs.
- Flexible CSPs relax constraints to handle overconstrained problems.
- Backtracking search is structurally based on a LIFO stack and recursion, and Prolog is a standard programming language for Constraint Programming.

9.6 Unit VI: Adversarial Search

9.6.1 1. Important Definitions

- **Adversarial Search:** Search algorithms used in competitive, multi-agent environments (games).
- **Minimax Algorithm:** A recursive algorithm that calculates optimal moves in zero-sum games by maximizing for MAX and minimizing for MIN.
- **Alpha-Beta Pruning:** An optimization that skips evaluating subtrees that cannot possibly affect the final minimax decision.

Alpha (α):* The best (highest) utility value found so far for MAX. Beta (β):* The best (lowest) utility value found so far for MIN.

9.6.2 Frequently Asked Questions

1. "Explain Alpha-Beta Pruning with a trace." [15M][[]]

Core point:* Explain α and β . Prune branches whenever $\beta \leq \alpha$ (alpha-cutoff at MIN node, beta-cutoff at MAX node). Show a sample tree with leaf evaluations and marked cutoffs.

9.6.3 Diagrams to Practice

1. Pruned Game Tree:

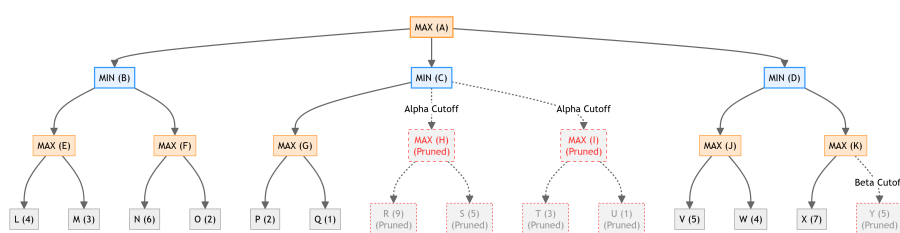


Figure 9.10: Alpha-Beta Pruning Game Tree

9.6.4 One-Line Revision

- Standard minimax has time complexity $O(b^d)$, which is reduced to $O(b^{d/2})$ with perfect alpha-beta pruning.
- Pruning does not change the optimal move selection; it only speeds up the search.

9.7 Unit VII: Logical Agents

9.7.1 1. Important Definitions

- **Propositional Logic:** A logic system dealing with propositions (T/F statements) and connectives ($\wedge, \vee, \neg, \rightarrow, \leftrightarrow$).
- **Soundness:** An inference rule is sound if it only derives true sentences from true premises.
- **Completeness:** An inference rule is complete if it can derive all sentences entailed by the premises.

- **Tautology:** A sentence that is True under all truth assignments (e.g., Peirce's Law: $((P \rightarrow Q) \rightarrow P) \rightarrow P$).
- **Modus Ponens:** Inference rule of form: $\frac{P, P \rightarrow Q}{Q}$.
- **Modus Tollens:** Inference rule of form: $\frac{\neg Q, P \rightarrow Q}{\neg P}$.
- **Unit Clause:** A clause containing exactly a single literal (e.g. P or $\neg Q$).
- **Proposition Symbol (AI Constants):** In AI, the two standard logical constant symbols representing fixed truth values: **True** ($\top$) and **False** ($\perp$).
- **Validity & Satisfiability:** A sentence is valid if it is true in all models (Tautology). A sentence is satisfiable if it is true in at least one model; if a sentence has no model where it is true, it is unsatisfiable (Contradiction).

9.7.2 Frequently Asked Questions

1. "Prove that Peirce's Law is a Tautology." [5M][[]]

Core point:* Construct a 4-row truth table for P, Q . Compute column-wise: $P \rightarrow Q$, $(P \rightarrow Q) \rightarrow P$, and the final implication, proving all rows are True.

1. "Explain Wumpus World environment." [10M][[]]

Core point:* State PEAS components. Show how Breeze maps to Pit and Stench to Wumpus, and trace safety reasoning.

9.7.3 Diagrams to Practice

1. Wumpus World Grid:

Wumpus World 4x4 Grid				
	4 Stench	Breeze	PIT	
	3 Agent	Breeze	PIT	Breeze
	2 Stench	Breeze		
	1 Start	Breeze	PIT	Breeze
	1	2	3	4

Figure 9.11: Wumpus World Grid Layout

9.7.4 One-Line Revision

- Knowledge-based agents query the KB with 'ASK' and insert facts/percepts with 'TELL'.
- Propositional logic is monotonic but lacks variables and quantifiers.

- Every sentence of propositional logic can be converted to an inferred equivalent CNF sentence. If the CNF is unsatisfiable, the original statement is unsatisfiable.
- Resolution is a single inference rule because it is refutation-complete, meaning no other rules (like Modus Ponens) are required to compute logical entailment.
- There are exactly 2 constant proposition symbols in AI logic: True and False.

9.8 Unit VIII: First-Order Logic & Knowledge Representation

9.8.1 1. Important Definitions

- **First-Order Logic (FOL):** Logic system representing objects, properties, relations, and quantifiers ($\forall, \exists$).
- **Horn Clause:** A clause containing at most one positive literal. Basis of PROLOG.
- **Skolemisation:** Eliminating existential quantifiers by replacing them with unique constants or functions (when inside universal scopes).
- **Semantic Network:** A graph with concept nodes and directed, labeled relation edges.
- **Fuzzy Logic:** A multi-valued logic representing degrees of membership in a set using real numbers in $[0, 1]$.

9.8.2 Frequently Asked Questions

1. "Convert a FOL formula into Clausal Form (CNF)." [15M][]

Core point:* Follow the 7 steps: eliminate implications, move negations inward, standardize variables, Skolemise, drop $\forall$, distribute $\vee$ over $\wedge$, isolate clauses.

1. "Prove Marcus hated Caesar using Resolution Refutation." [15M][]

Core point:* Translate statements into FOPL clauses. Negate the goal ($\neg Hate(Marcus, Caesar)$). Apply resolution to derive the empty clause ($\square$).

9.8.3 Diagrams to Practice

1. Semantic Net Diagram:

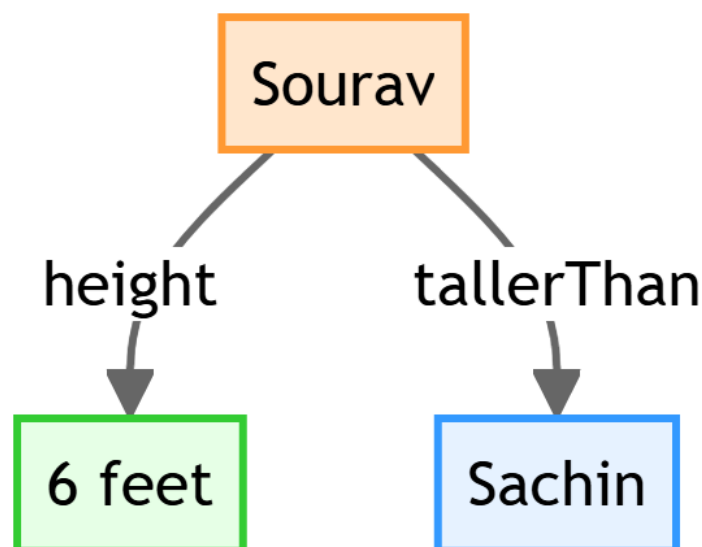


Figure 9.12: Semantic Network

1. Resolution Refutation Proof Tree:

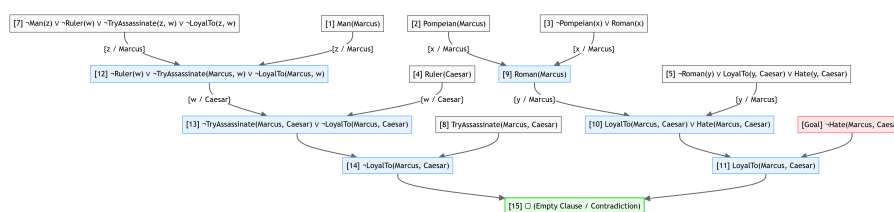


Figure 9.13: Resolution Refutation Proof Tree

9.8.4 One-Line Revision

- Fuzzy union uses max and intersection uses min of membership functions.
- Hebbian learning in NN adjusts weights according to $\Delta w_{ij} = \eta a_i a_j$.
- Genetic algorithms search using populations, selection, crossover, and mutation.

9.9 Solved Exam One-Liners (CA4 Database)

- **Q35.** Which are needed to compute the logical inference algorithm?

Answer:* **Validity, Satisfiability, and Logical equivalence** (All of the mentioned)

- **Q36.** Which search method will expand the node that is closest to the goal?

Answer:* **Greedy best-first search** (evaluates using $f(n) = h(n)$)

- **Q37.** Which search uses the problem specific knowledge beyond the definition of the problem?

Answer:* **Informed search** (or Heuristic search)

- **Q38.** Backtracking is based on which data structure and algorithm mechanics?

Answer:* **Last-In, First-Out (LIFO) stack and Recursion (Both)**

- **Q39. How many proposition symbols are there in artificial intelligence logic?**

Answer:* **2** (representing the constant truth symbols **True** and **False**)

- **Q40. Which sentences can be converted to inferred equivalent CNF sentences?**

Answer:* **Every sentence of propositional logic**

- **Q41. What can be viewed as a single literal of disjunction?**

Answer:* **Unit clause**

- **Q42. Uninformed search strategies are better than informed search strategies.**

Answer:* **False** (informed searches use heuristics to dramatically reduce search space)

- **Q43. Which values are independent in minimax search algorithm?**

Answer:* **Pruned leaves x and y** (their values do not affect the root decision)

- **Q44. Which sentence will be unsatisfiable if the CNF sentence is unsatisfiable?**

Answer:* **Original statement** *Q45. The A algorithm is based on which search strategy?*

Answer:* **Best-First Search** (using evaluation function $f(n) = g(n) + h(n)$)

- **Q46. Flexible CSPs relax on which component of the problem?**

Answer:* **Constraints** (soft constraints/preferences are relaxed to resolve overconstrained scenarios)

- **Q47. Which is used to compute the truth of any sentence?**

Answer:* **Semantics of propositional logic** (evaluates truth values in a model)

- **Q48. Preparing a schedule for a class of students is what type of problem?**

Answer:* **CSP (Constraint Satisfaction Problem)**

- **Q49. Which search is complete and optimal when the heuristic $h(n)$ is consistent?**

Answer: **A search** (optimal for graph search with consistent heuristics)

- **Q50. The Heuristic function $h(n)$ represents: -----.**

Answer:* **Estimated cost of the cheapest path from the current node to the goal node**

- **Q51. Which form is called a conjunction of disjunction of literals?**

Answer:* **Conjunctive Normal Form (CNF)**

- **Q52. What is the space complexity of Greedy best-first search?**

Answer:* $O(b^l)$ (or $O(b^m)$), which is exponential in the worst case

- **Q53. Which inference rule is also called a single inference rule?**

Answer:* **Resolution** (refutation-complete rule)

- **Q54. Which of the following problems can be modeled as a CSP?**

Answer:* **8-Puzzle, Map coloring, and 8-Queen problems** (All of the mentioned)

- **Q55. Which search uses only linear space for searching?**

Answer: ***Recursive Best-First Search (RBFS)*** (*uses $O(bd)$ space, unlike A's exponential space*)

- **Q56. Constraint satisfaction problems on finite domains are typically solved using: -----.**

Answer:* **Search Algorithms** (specifically Backtracking Search)

- **Q57. The search strategy that uses problem-specific knowledge is known as: -----.**

Answer:* **Informed Search** (or Heuristic Search)

- **Q58. Best-First search can be implemented using which data structure?**

Answer:* **Priority Queue** (keeps frontier sorted by evaluation function $f(n)$)

- **Q59. Which programming language is used for Constraint Programming?**

Answer:* **Prolog** (with specialized CLP(FD) constraint solver libraries)